Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Jeu des Amazones – Projet C

Rapport final

Nicolas Caminade, Thomas Moreau, Tanguy Decabrat, Etudiant4, Etudiant5

10 avril 2026

1 Introduction

Le jeu des Amazones est un jeu de société combinatoire abstrait inventé par l'Argentin Walter Zamkauskas en 1988 et publié pour la première fois dans la revue *El Acertijo* en 1992 [1]. Il se joue traditionnellement sur un plateau de 10×10 cases et combine des éléments des échecs (le déplacement de la dame) et du jeu de Go (le contrôle du territoire).

1.1 Historique et héritage

Né en Argentine à la fin des années 1980 sous la plume de **Walter Zamkauskas** [1], le **Jeu des Amazones** a d'abord circulé dans des cercles restreints d'amateurs de jeux abstraits[1]. Sa véritable diffusion internationale s'opère en 1992 grâce à la publication de ses règles dans la revue *El Acertijo*. Très rapidement, sa complexité conceptuelle qui mêle la liberté de déplacement des **échecs** à la notion de **blocage territorial** du jeu de **Go** attire l'attention bien au-delà de la sphère ludique traditionnelle. Dès la fin des années 1990, le monde académique s'empare de ce jeu, fasciné par son **facteur de branchement** qui, contrairement à d'autres jeux de plateau, diminue drastiquement au fil de la partie à mesure que l'espace se réduit.

Au fil des années, les Amazones se sont imposées comme un banc d'essai incontournable pour l'informatique théorique et l'intelligence artificielle. En 2005, l'équipe de Furtak démontre mathématiquement que la résolution générale de ce jeu appartient à la classe des problèmes **PSPACE-complets** [2], confirmant définitivement qu'il résiste à toute approche par force brute. Face à ce défi, la communauté scientifique a multiplié les expérimentations. Richard Lorentz a notamment marqué l'histoire du jeu avec son programme *Invader*, prouvant que l'approche statistique du **Monte Carlo Tree Search (MCTS)** permettait d'apprivoiser cette explosion combinatoire [3]. Grâce à ces travaux, le Jeu des Amazones est devenu un véritable cas d'école pour l'évaluation des heuristiques de recherche arborescente.

Aujourd'hui, sur le plan de la recherche, il sert de terrain d'entraînement privilégié pour les algorithmes modernes. Après l'intégration du **Deep Learning** pour modéliser une « intuition » artificielle capable de rivaliser avec le MCTS, les travaux les plus récents exploitent désormais les grands modèles de langage (**LLMs**) et les réseaux d'attention sur les graphes. En 2026, Qian et ses collègues ont par exemple démontré comment un modèle hybride pouvait apprendre à partir de données bruitées générées par une IA généraliste (comme GPT-4) pour prendre des décisions hautement spécialisées et performantes, et ce même dans des environnements aux ressources de calcul limitées [4]. Sur le plan compétitif, le jeu s'est illustré pendant de nombreuses années comme une épreuve classique des **Olympiades Informatiques** (*Computer Olympiad*), où les meilleurs moteurs du monde s'affrontent. Enfin, auprès du grand public, le jeu a trouvé un second souffle grâce au numérique. Il est désormais intégré à de nombreuses plateformes en ligne dédiées aux jeux abstraits de réflexion[2], rassemblant une communauté active de joueurs humains qui continuent de se mesurer à la machine et d'en explorer sa profondeur stratégique.

1.2 Règles

Le Jeu des Amazones se pratique sur un damier, traditionnellement de 10×10 cases. Chaque joueur dispose de quatre amazones, disposées symétriquement en début de partie selon une configuration initiale précise (voir la **Figure 1a**) : les Blancs placent leurs pièces en (d1), (g1), (a4) et (j4), tandis que les Noirs occupent les cases (d10), (g10), (a7) et (j7). Cette disposition garantit un équilibre stratégique parfait entre les deux camps.

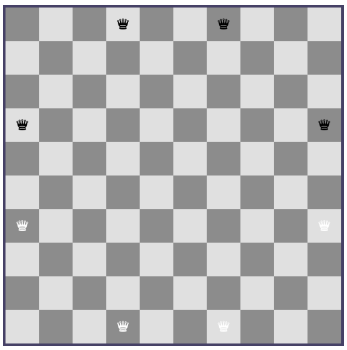
1. Les règles complètes et l'historique ludique sont largement documentés sur des bases de données comme BoardGameGeek : `https://boardgamegeek.com/boardgame/2125/amazons`
2. Des plateformes incontournables comme Little Golem (`https://www.littlegolem.net/`) pour le tour par tour asynchrone, ou Tabletopia (`https://tabletopia.com/games/amazons`) pour la simulation 3D du plateau.

Objectif du jeu. La victoire revient au dernier joueur capable d'effectuer un coup légal. L'enjeu est donc de priver progressivement l'adversaire de tout espace de déplacement en combinant occupation du terrain et tirs de flèches. Contrairement aux jeux comme les échecs ou le Go, il n'existe pas de capture de pièce : le matériel reste constant tout au long de la partie.

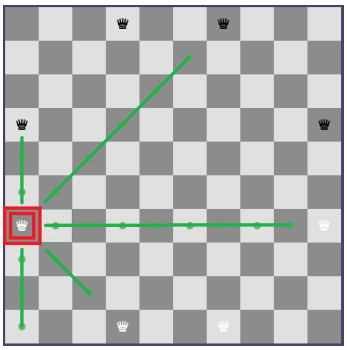
Déroulement d'un tour. Chaque tour est impérativement composé de deux phases successives et indissociables. Dans un premier temps, le joueur choisit l'une de ses amazones et la déplace en ligne droite — verticalement, horizontalement ou en diagonale — vers une case libre (**Figure 1b**). Ce mouvement est identique à celui de la dame aux échecs : l'amazone peut parcourir autant de cases qu'elle le souhaite dans la direction choisie, mais elle ne peut ni traverser ni s'arrêter sur une case occupée par une autre pièce (amie ou ennemie) ou par une flèche. Dans un second temps, depuis sa nouvelle position, cette même amazone doit obligatoirement tirer une flèche (**Figure 1c**). La flèche obéit aux mêmes règles de trajectoire que l'amazone : elle se déplace en ligne droite dans l'une des huit directions possibles, sans pouvoir franchir d'obstacle. La case où la flèche atterrit est définitivement brûlée et inaccessible pour le reste de la partie (**Figure 1d**).

Subtilités tactiques. Une règle souvent mal comprise par les débutants concerne le tir de la flèche : l'amazone peut tirer dans n'importe quelle direction autorisée depuis sa case d'arrivée, y compris en direction de la case qu'elle vient de quitter. Cette subtilité ouvre des possibilités tactiques riches, notamment la possibilité de s'auto-bloquer temporairement pour mieux circonscrire l'adversaire. De plus, la flèche ne peut traverser une case occupée par une amazone adverse, ce qui signifie que les pièces ennemies servent d'obstacles naturels aux tirs.

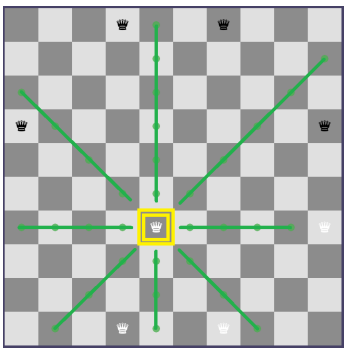
Fin de partie et variantes. Le jeu des Amazones ne connaît pas de match nul par répétition ou par épuisement de matériel : puisque chaque tour réduit irrémédiablement le nombre de cases disponibles, la partie se termine forcément par le blocage complet d'un joueur. En pratique, les parties se divisent souvent en deux phases distinctes : une ouverture stratégique où les amazones s'affrontent pour le contrôle des grandes zones du plateau, suivie d'une phase finale où chaque joueur maximise son territoire cloisonné. Bien que le format 10×10 soit le standard, des variantes sur plateau 6×6 ou 8×8 existent avec un nombre d'amazones adapté, permettant des parties plus rapides tout en conservant la richesse tactique du jeu.



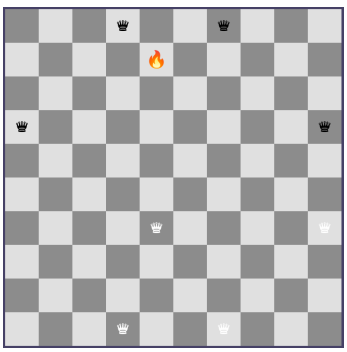
(a) Position initiale



(b) Sélection de la reine : on sélectionne un nouvel emplacement pour la reine



(c) Placement de la flèche : une fois la reine placée, on tire sur un nouvel endroit



(d) Flèche placée

FIGURE 1 – Évolution d'une partie : position initiale et premier coup

1.3 Complexité

Le jeu des Amazones est d'une très grande complexité [5]. Le tableau ci-dessous emploie les abréviations Branch. pour « facteur de branchement » et Cmplx. pour « complexité ».

	Échecs (8×8)	**Amazones (10×10)**	**Go (19×19)**
Facteur Branch. Initial	~ 20	~ 2176	~ 361
Facteur Branch. Moyen	~ 35	~ 374 joueur de départ ~ 299 deuxième joueur ~ 50 durant la fin de partie	~ 250
Nombre de tours moyen	~ 80	~ 84	~ 150
Cmplx. Arbre de jeu	$\sim 10^{123}$	$\sim 1.09 * 10^{212}$	$\sim 10^{360}$
Cmplx. Espaces d'états	de $\sim 3.7 * 10^{43}$ à $\sim 10^{52}$	$\sim 6.45* 10^{40}$	$\sim 10^{170}$

TABLE 1 – Comparaison des complexités pour des tailles de plateau traditionnelles (8×8 pour les échecs, 10×10 pour les Amazones, 19×19 pour le Go) [6][7]

Comme l'illustre ce tableau comparatif, la complexité des Amazones réside dans son facteur de branchement initial extrême, surpassant largement celui des échecs et comparable à celui du Go. Bien que la profondeur de jeu soit modérée, cette « largeur » excessive génère un arbre de jeu colossal (10^{212}), rendant toute exploration exhaustive impossible en raison de l'explosion combinatoire. Pour contourner cette explosion, les approches modernes s'appuient sur la sélectivité : solveurs de preuve comme *df-pn* [8], algorithmes de jeu utilisant du *Forward Pruning* [9], méthodes statistiques comme le MCTS via l'Upper Confidence bounds applied to Trees (UCT) [10], ou encore le Machine Learning [4].

2 Algorithmes et structures de données

2.1 Moteur de jeu : algorithmes

2.1.1 Validation physique des déplacements (Algorithme de Raycasting)

La règle principale des Amazones stipule qu'aucune pièce ne peut franchir un obstacle. Pour valider mathématiquement un coup saisi par un joueur, le moteur implémente un algorithme de vérification fonctionnant sur le principe du lancer de rayon (*Raycasting*) en grille discrète.

L'algorithme calcule d'abord le vecteur de déplacement $\vec{v} = (\Delta x, \Delta y)$ entre le point de départ $A(x_1, y_1)$ et le point d'arrivée $B(x_2, y_2)$, puis vérifie que le mouvement respecte la contrainte de la dame aux échecs, c'est-à-dire que $\Delta x = 0$, ou $\Delta y = 0$, ou $|\Delta x| = |\Delta y|$. Il détermine ensuite un pas géométrique valant -1, 0 ou 1 pour chaque axe, via la formule $step_x = \text{sign}(\Delta x)$ et $step_y = \text{sign}(\Delta y)$. Une boucle itérative parcourt alors le segment point par point : si la carte globale des obstacles révèle une collision à la coordonnée courante, l'exploration s'interrompt prématurément et la trajectoire est déclarée invalide.

2.1.2 Génération des coups possibles

Le Jeu des Amazones se distingue par un facteur de branchement élevé. Contrairement aux échecs, une action n'est pas unitaire, mais séquentielle et interdépendante : le déplacement d'une reine modifie immédiatement la topologie des obstacles pour le tir de la flèche qui suit.

Pour fournir à l'intelligence artificielle l'ensemble exhaustif des coups légaux, le moteur implémente un algorithme de génération de l'arbre des possibles basé sur une exploration combinatoire contrainte. Cet algorithme opère en trois phases déterministes :

Il isole le sous-ensemble Q contenant les positions des quatre reines du joueur actif. Ensuite pour chaque reine $q \in Q$, l'algorithme effectue un balayage spatial omnidirectionnel pour déterminer D_Q, l'ensemble de ses cases de destination atteignables. Pour chaque destination candidate $d \in D_Q$, l'algorithme simule virtuellement le déplacement. Cette translation modifie l'état du plateau : la case d'origine de la reine devient libre, et sa case d'arrivée devient un obstacle. Depuis cette nouvelle position d, une seconde expansion spatiale est calculée pour obtenir D_A, l'ensemble des cibles valides pour la flèche.

Le nombre total de coups possibles à un instant donné se calcule en additionnant toutes ces combinaisons. Cela se modélise par la formule mathématique suivante :

$$\text{Total des coups} = \sum_{q \in Q} \sum_{d \in D_Q} |D_A(d)|$$

Afin d'expliquer ce calcul plus simplement : Q représente le groupe des quatre reines du joueur actif. Pour chaque reine q, l'ensemble D_Q désigne toutes les cases de destination qui lui sont accessibles. Pour chaque destination d choisie, la notation $|D_A(d)|$ représente simplement le nombre exact de flèches qu'il est possible de tirer depuis cette nouvelle case.

Cette double somme illustre bien la mécanique « à deux temps » propre au jeu des Amazones. Aux échecs, le déplacement d'une pièce (comme un fou ou une tour) demande un calcul simple avec une complexité linéaire $\mathcal{O}(R)$, où R est la distance maximale de déplacement. Dans notre moteur, le fait de lier le mouvement et le tir nous oblige à multiplier ces possibilités entre elles, élevant la complexité d'exploration à $\mathcal{O}(R^2)$ par pièce. C'est précisément pour supporter ce très grand volume de calculs que l'utilisation des structures *Bitboards* s'est imposée pour l'Intelligence Artificielle.

2.2 Le cœur du jeu et l'Intelligence Artificielle

2.2.1 Représentation de l'état et Complexité : les Bitboards

Pour garantir des performances optimales, nous avons modélisé le plateau via des *Bitboards* [11]. Le plateau pouvant atteindre une taille de 11×11 (121 cases), une simple variable de 64 bits était insuffisante. Nous avons donc conçu une structure `Bitboard` composée de deux entiers non signés ($2\times$ `uint64_t`), offrant 128 bits d'espace.

Cette architecture matérielle permet d'exécuter la majorité des opérations classiques en temps constant $\mathcal{O}(1)$, comme les manipulations de masques (ET, OU, XOR) ainsi que les décalages de bits. Le calcul du débordement entre nos deux entiers 64 bits (`low` et `high`) ajoute un surcoût mineur, mais la complexité temporelle globale reste strictement bornée et indépendante de l'état du plateau.

La véritable puissance de cette structure, par rapport aux approches matricielles classiques, s'exprime lors de la génération des trajectoires (*slides*), qui s'effectue en temps constant global $\mathcal{O}(1)$. Dans nos heuristiques, nous distinguons deux types de déplacements :

- Le ***King step*** : il simule une avancée d'une seule case dans les huit directions, pour savoir par exemple si une reine peut bouger. Il se calcule par de simples décalages de bits immédiats.
- Le ***Queen slide*** : il calcule la portée maximale d'une pièce avant de rencontrer un obstacle (le bord du plateau ou une autre pièce).

De manière naïve, calculer un *Queen slide* nécessite des boucles itératives conditionnelles (avancer d'une case, vérifier si elle est vide, continuer, sinon arrêter). Ces boucles sont extrêmement coûteuses en temps d'exécution pour le processeur (étant appelées à chaque évaluation).

La solution : l'algorithme de Kogge-Stone. Pour éliminer totalement ces boucles, nous avons implémenté l'algorithme de Kogge-Stone [12]. Issu de l'architecture matérielle des additionneurs, cet algorithme repose sur le principe de propagation parallèle (*Parallel Prefix*). Au lieu d'avancer case par case, le processeur « pousse » simultanément l'ensemble des pièces d'une même couleur à travers un masque représentant les cases vides de la grille.

Mécanique des décalages exponentiels. Cette propagation s'effectue via une séquence stricte de quatre décalages bit-à-bit exponentiels. À chaque étape, la zone d'influence de la pièce double sa portée en s'appuyant sur le résultat de l'étape précédente. Pour illustrer ce concept, prenons une ligne de 8 cases lue de gauche à droite. L'amazone part de la première case, et un obstacle (une flèche ou une autre pièce) occupe la sixième case :

- **État initial :**
  `G = 1000 0000` (La pièce est sur le bit de poids fort)
  `P = 0111 1011` (Les cases vides : le premier '0' est la reine, le deuxième un mur)
- **Décalage de 1 :** La pièce s'étend sur 1 case adjacente vide. Le masque se décale lui aussi pour repérer les futurs blocs de 2 cases vides consécutives. (Il projete une "ombre" de taille équivalente au prochain décalage, pour ne pas pouvoir sauter par dessus cet obstacle)
  `G = 1100 0000` (La portée est maintenant de 2 cases)
  `P = 0011 1001` (Mise à jour via un ET logique avec lui-même décalé de 1)
- **Décalage de 2 :** La pièce bondit de 2 cases d'un coup en s'appuyant sur le nouveau masque. Le masque repère les blocs de 4 cases vides.
  `G = 1111 0000` (La portée continue de s'étendre)
  `P = 0000 1000` (Plus qu'un seul emplacement valide pour un futur grand bond)
- **Décalage de 4 :** La pièce tente un bond de 4 cases, mais elle est bloquée par la raréfaction des '1' dans le masque.
  `G = 1111 1000` (L'expansion s'arrête exactement *devant* l'obstacle)
  `P = 0000 0000` (Le masque se vide totalement)

— **Décalage de 8 :** Ultime étape pour les plateaux étendus, permettant de couvrir jusqu'à 15 cases (sans effet ici puisque `P` ne contient plus que des zéros).

Puisque la distance maximale franchissable sur notre plateau est de 10 cases (sur une grille 11×11), cette séquence couvre l'intégralité des trajectoires possibles. Ainsi, ce qui nécessiterait des dizaines d'itérations conditionnelles dans une matrice classique est ici résolu de manière purement mathématique et simultanée en seulement quatre instructions.

Enfin, la seule complexité variable de notre modèle intervient lors de l'extraction des coups ou de l'évaluation des heuristiques. Bien que l'instruction matérielle d'extraction (*Pop Least-Significant-Bit* ou `pop_lsb`) s'exécute instantanément en $\mathcal{O}(1)$, elle est invoquée au sein d'une boucle pour lister les éléments actifs. Cette boucle s'exécutant exactement k fois (où k est le nombre strict de pièces ou cases évaluées), nous garantissons une complexité d'itération en $\mathcal{O}(k)$. Ce comportement est infiniment plus performant qu'un parcours complet de la grille en $\mathcal{O}(N)$ inhérent aux tableaux bidimensionnels.

2.2.2 L'approche Minimax et ses heuristiques

Le premier moteur d'IA repose sur l'algorithme Minimax, optimisé par l'élagage Alpha-Beta [13]. Pour pallier l'explosion combinatoire, nous avons enrichi cette base par deux mécanismes paramétrables. L'approfondissement itératif (*Iterative Deepening*) garantit le renvoi du meilleur coup trouvé à la profondeur maximale complétée si la limite de temps est atteinte. L'élagage anticipé (*Forward Pruning* [9]) restreint la largeur du faisceau de recherche en ne conservant que les coups offrant le plus de contrôle de plateau. Sans ces optimisations, le système bascule de manière transparente sur un Alpha-Beta exhaustif classique.

L'évaluation des nœuds feuilles constitue le cœur décisionnel de notre stratégie. Nous avons conçu et comparé trois fonctions d'évaluation offrant un compromis graduel entre rapidité et finesse mathématique.

La première approche est l'évaluation de mobilité [5], axée sur la mobilité relative (`eval_me`), calcule rapidement la différence entre le nombre total de coups légaux disponibles pour les Blancs et les Noirs. En simulant une expansion de type *Queen slide* pour toutes les reines, l'algorithme dénombre les cases uniques atteignables. Soit M_p l'ensemble des cases atteignables par le joueur p, l'heuristique est définie par :

$$\mathcal{H}_{me} = |M_{blanc}| - |M_{noir}|$$

Cette légèreté calculatoire permet d'atteindre d'excellentes profondeurs d'exploration, mais l'évaluation demeure myope et peine à anticiper la fermeture structurelle des territoires.

La deuxième heuristique, dite de territoire relatif (`eval_rth`) [5], évalue le contrôle spatial en déployant un parcours en largeur (BFS) simultané. Elle calcule la distance, en nombre de mouvements de Reine, séparant chaque case vide de la pièce la plus proche. Soit $\mathcal{E}$ l'ensemble des cases vides, et $d_W(x)$ (resp. $d_B(x)$) la distance minimale entre une case x et une reine blanche (resp. noire) :

$$\mathcal{H}_{rth} = \sum_{x \in \mathcal{E}} \Delta(x) \quad \text{avec} \quad \Delta(x) = \begin{cases} \min(d_B(x), C) - d_W(x) & \text{si } d_W(x) < d_B(x) \\ -(\min(d_W(x), C) - d_B(x)) & \text{si } d_B(x) < d_W(x) \\ 0 & \text{sinon} \end{cases}$$

Une distance infinie est plafonnée par une constante C (définie par $C =$ taille du plateau $+ 2$).

Notre modèle le plus abouti repose sur les territoires absolus combinés (`eval_qdkd`) [14]. Il simule l'appropriation du plateau par une expansion continue (*flood-fill*) traitée case par case. La force de cette heuristique réside dans sa résolution des conflits : si une case est atteinte par les deux joueurs à la même itération k, elle est déclarée contestée et agit comme un obstacle infranchissable. Soient $T_Q(p)$ et $T_K(p)$ les ensembles de cases conquises par le joueur p selon des déplacements respectifs de Reine et de Roi :

$$t_1 = |T_Q(Blanc)| - |T_Q(Noir)| \qquad t_2 = |T_K(Blanc)| - |T_K(Noir)|$$

Le calcul final applique une pondération mariant vision globale et contrôle local (choisi d'après les pistes de calcul de Lieberum [14]) :

$$\mathcal{H}_{qdkd} = 1.0 \times t_1 + 0.5 \times t_2$$

2.2.3 Monte Carlo Tree Search (MCTS) et apprentissage automatique

Le second moteur d'IA exploite le Monte Carlo Tree Search. Pour modéliser l'arborescence, nous avons opté pour une structure de données **LCRS** (*Left-Child Right-Sibling*) [15]. Ce choix découle directement du facteur de branchement initial supérieur à 2 000 coups possibles : utiliser des tableaux de pointeurs pour stocker les enfants de chaque nœud aurait provoqué une explosion de l'empreinte mémoire. Le choix de la structure LCRS (*Left-Child Right-Sibling*) répond à une contrainte majeure du jeu des Amazones : son facteur de branchement exceptionnellement élevé (entre

500 et 2 000 coups possibles par tour). Une approche classique, où chaque nœud pré-allouerait un tableau pointant vers tous ses enfants potentiels, saturerait la mémoire système de manière quasi instantanée.

À l'inverse, l'approche LCRS garantit une empreinte mémoire minimale et fixe. Chaque nœud ne contient que deux pointeurs : l'un vers son premier enfant, l'autre vers son frère immédiat. Cette optimisation spatiale implique un compromis temporel, car l'accès au k-ième enfant nécessite un parcours séquentiel en $O(k)$ cycles CPU.

Cependant, ce coût est largement justifié. Couplée à une instanciation dynamique des enfants — où un nœud n'est créé que lorsque le coup correspondant est réellement exploré — cette architecture empêche l'explosion mémoire tout en garantissant l'efficacité du moteur. Nous avons également enrichi ce MCTS d'un réseau de neurones. Dans ce mode, l'évaluation des nœuds non terminaux est combinée à l'inférence d'un modèle TensorFlow recevant en entrée un tenseur à quatre canaux généré par nos bitboards.

Politique de sélection UCT standard. Dans sa version classique, le MCTS guide la phase de sélection de l'arborescence à l'aide de la formule **UCT** (*Upper Confidence bounds applied to Trees*) [10]. Cette métrique applique les principes des bandits manchots à la recherche arborescente afin de trouver le compromis mathématique optimal entre l'exploitation des branches prometteuses et l'exploration des chemins peu visités. Pour un nœud enfant n, l'évaluation s'écrit :

$$\mathrm{UCT}(n) = Q(n) + c \cdot \sqrt{\frac{\ln(N(parent))}{N(n)}}$$

où $Q(n)$ représente la valeur moyenne empirique du nœud (le taux de victoire obtenu par les simulations aléatoires), $N(parent)$ et $N(n)$ désignent respectivement le nombre total de visites du nœud parent et de l'enfant évalué, et c est la constante d'exploration (théoriquement fixée à $\sqrt{2}$). Le premier terme favorise l'exploitation pure, tandis que le second garantit une exploration continue : la racine carrée croît de manière logarithmique avec les visites du parent, forçant l'algorithme à évaluer les enfants délaissés (où $N(n)$ est faible ou nul).

Variante PUCT pour l'apprentissage automatique. Lorsque le réseau de neurones est activé, la politique de sélection UCT classique est remplacée par la variante **PUCT** (*Predictor + Upper Confidence Bound*) [16]. Cette approche hybride combine l'exploration statistique d'UCT avec une évaluation par réseau de neurones pour guider plus efficacement la recherche.

La formule PUCT pour un nœud enfant n s'écrit :

$$\mathrm{PUCT}(n) = \frac{Q_{\text{rollout}}(n) + Q_{\text{NN}}(n)}{2} + c_{\text{puct}} \cdot \frac{\sqrt{N(parent)}}{1 + N(n)}$$

où $Q_{\text{rollout}}(n)$ est le taux de victoire empirique obtenu par les simulations Monte Carlo, $Q_{\text{NN}}(n)$ est la probabilité de victoire prédite par le réseau de neurones, $N(parent)$ et $N(n)$ sont respectivement les nombres de visites du parent et de l'enfant, et c_{puct} est une constante d'exploration (par défaut 1.5).

Le réseau de neurones utilisé est un modèle de régression linéaire simple entraîné sur des parties auto-générées. Il reçoit en entrée un tenseur de dimension $11 \times 11 \times 4$ représentant :

— Canal 0 : positions des reines blanches (1 si présente, 0 sinon)
— Canal 1 : positions des reines noires
— Canal 2 : positions des flèches
— Canal 3 : masque de validité du plateau (1 pour les cases valides)

Cette architecture à quatre canaux permet au modèle de capturer simultanément la configuration matérielle et la topologie du plateau. Lors de la sélection, si l'évaluation neuronale d'un nœud n'a pas encore été calculée (`ai_eval` ≤ -2.0), le tenseur correspondant est généré à la volée et soumis au modèle TensorFlow. La valeur retournée est ensuite mise en cache dans le nœud pour éviter les inférences redondantes.

2.3 Réseau : structures de données et protocole

La mise en place du mode multijoueur a nécessité de résoudre deux contraintes fondamentales de la programmation réseau : la nature orientée flux du protocole TCP et la nécessité de maintenir un état cohérent pour chaque connexion asynchrone.

2.3.1 Le problème du *Framing* TCP

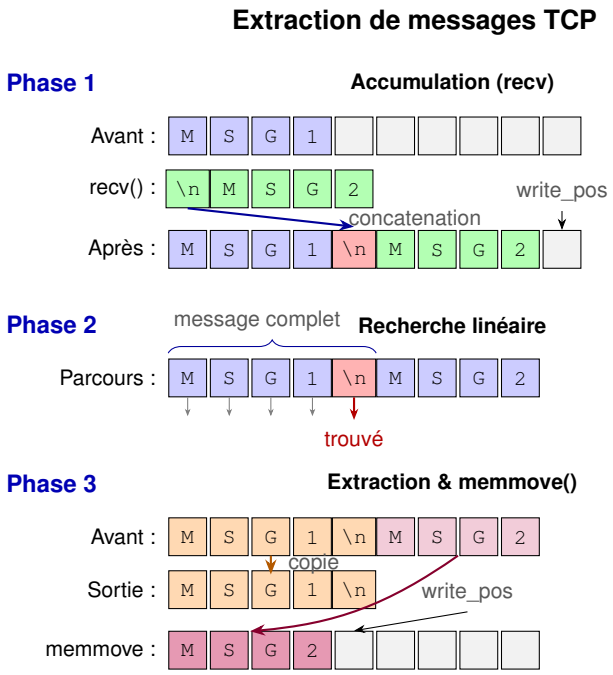
Contrairement à UDP qui fonctionne par datagrammes délimités, TCP est orienté flux : l'émetteur peut envoyer trois messages de 100 octets, mais le récepteur pourrait recevoir un bloc de 250 octets suivi d'un fragment de 50 octets. Ce phénomène, appelé *TCP Framing*, rend impossible la lecture directe de messages individuels sans mécanisme de délimitation.

Nous avons donc opté pour un délimiteur de fin de message (`'\n'`) plutôt qu'un préfixe de longueur. Ce choix présente deux avantages : il permet le débogage à l'œil nu via des outils standards (`netcat`, `tcpdump`), et il évite

les problèmes de désynchronisation si un octet est corrompu, une erreur dont la récupération serait impossible sans fermer la connexion.

Pour accumuler les fragments TCP jusqu'à réception d'un message complet, chaque connexion dispose de son propre tampon linéaire de 4 096 octets. Ce dimensionnement (environ 2,7 fois le MTU Ethernet standard de 1 500 octets) garantit qu'au moins deux messages complets peuvent être mis en file simultanément, absorbant ainsi les pics de trafic sans saturation.

L'extraction des messages s'effectue en trois phases distinctes. L'**accumulation** consiste en la concaténation à la fin du tampon des octets bruts reçus via `recv()`. Ensuite avec la **recherche** linéaire on localise le premier caractère de fin de ligne dans le tampon. Enfin le message extrait est **copié** vers un tampon de sortie, puis les octets restants sont décalés vers le début du tampon via `memmove()`, préservant ainsi les fragments du message suivant. Cette dernière opération, bien que coûteuse en temps ($\mathcal{O}(n)$ où n est la taille des données restantes), reste négligeable face aux latences réseau et garantit une gestion mémoire sans fragmentation.



2.3.2 Protocole applicatif et sérialisation

Au-dessus de la couche de framing, nous avons défini un protocole applicatif textuel structuré autour d'une unité atomique : le `Packet`.

Le format ASCII a été préféré au binaire pour trois raisons. Premièrement, il élimine les problèmes de l'ordre dans lequel les octets sont placés lors de la communication entre architectures hétérogènes (x86, ARM). Deuxièmement, il facilite le débogage sans outils spécialisés : un simple `netcat` suffit à observer les échanges. Troisièmement, il permet une évolution du protocole par ajout de nouveaux types de messages sans rétrocompatibilité binaire complexe.

Chaque paquet suit le format `"TYPE|PAYLOAD\n"`, où `TYPE` est un entier identifiant le type de message, le caractère `'|'` sert de séparateur unique entre le type et la charge utile, et le caractère `'\n'` permet à la couche de framing de détecter la fin du message. Cette conception garantit un parsing déterministe en $\mathcal{O}(m)$ où m est la longueur du message.

Le protocole définit 27 types de paquets répartis en trois catégories : les messages client vers serveur (préfixés `C2S_`), les messages serveur vers client (préfixés `S2C_`), et les messages bidirectionnels. Le tableau 23 en annexe présente l'ensemble des types avec leur rôle respectif.

2.4 Moteur de jeu : structures de données

Afin de garantir un moteur de jeu rapide et évolutif tout en anticipant les besoins de calcul intensif de l'intelligence artificielle, une attention particulière a été portée à la conception des structures de données de bas niveau.

2.4.1 Modélisation matricielle de l'état du jeu

L'état global de la partie est encapsulé dans une structure unique `GameState`. La représentation structurelle de base du plateau est stockée sous forme d'un tableau dynamique à deux dimensions (matrice de type `CellContent** board`). Chaque cellule contient une énumération définissant son état (`CONTENT_EMPTY`, `CONTENT_WHITE_QUEEN`, `CONTENT_ARROW`, etc.). Cette modélisation matricielle offre une lisibilité optimale pour l'affichage et permet un accès spatial en temps constant $\mathcal{O}(1)$. Cependant, elle montre de fortes limites pour l'exploration de l'arbre par l'IA : le parcours itératif d'une matrice 2D deviendrait un goulot d'étranglement lors des millions de simulations nécessaires, d'où le recours aux bitboards décrits plus haut.

2.4.2 Architecture Bitboard

Le moteur maintient et consolide l'état du jeu à travers trois `Bitboards` distincts, extraits dynamiquement depuis le `GameState` : `bb_white` pour la position des reines blanches, `bb_black` pour les reines noires, et `bb_arrows` pour l'ensemble des flèches tirées. Cette représentation secondaire parallèle est indispensable pour l'IA et les fonctions d'évaluation, tandis que la représentation matricielle reste utilisée pour l'affichage et la validation unitaire des règles.

2.5 Interface graphique : structures de données

L'interface graphique s'appuie sur la boîte à outils GTK4 [17], qui suit un paradigme orienté objet.

2.5.1 Le contexte applicatif `GuiContext`

De façon symétrique à l'architecture de la couche terminal, toute l'information d'état de l'interface graphique est regroupée dans un objet global `GuiContext`. Ce patron d'encapsulation (*Context Object*) évite les variables globales et facilite le passage d'état entre les différents gestionnaires d'événements.

Cette structure centralise tous les sous-systèmes : la configuration du jeu, l'état courant de la partie (`GameState`), le conteneur principal de la fenêtre, mais également le moteur d'IA et le gestionnaire réseau (`NetworkManager`). Le regroupement de ces instances permet au contrôleur graphique d'orchestrer des parties locales, contre l'IA, ou en multijoueur réseau de façon transparente.

Le plateau de jeu visuel est représenté par un tableau unidimensionnel de conteneurs de taille $n \times n$. Ce choix d'aplatissement (indexé par `y * size + x`) simplifie les calculs de conversion de coordonnées matricielles et évite la fragmentation en mémoire.

2.5.2 L'automate des phases de jeu

Un tour aux Amazones est indivisible : déplacer une reine puis tirer une flèche constitue une seule action logique. Pour modéliser cette contrainte dans l'interface graphique, nous avons défini une machine à états finis régie par deux statuts : `PHASE_MOVE_QUEEN` et `PHASE_SHOOT_ARROW`. Ce mécanisme garantit que l'interface est un automate déterministe : un événement de clic ou de glisser-déposer n'est traité que si l'état courant correspond à l'action attendue par les règles du jeu.

2.5.3 Les raccourcis clavier `KeyBindings`

Une structure `KeyBindings` stocke sous forme de chaînes de caractères (au format d'accélérateurs logiciels, ex : `"<Control>n"`) les raccourcis clavier de l'application, modifiables en direct. À chaque modification, le contrôleur de raccourcis de la fenêtre est dynamiquement détruit puis recréé, garantissant une cohérence stricte entre la vue des paramètres et le déclenchement des actions.

3 Architecture du projet

3.1 Découpage en modules

L'application suit une architecture en couches respectant strictement le principe de séparation des responsabilités. La figure 2 illustre les dépendances entre modules.

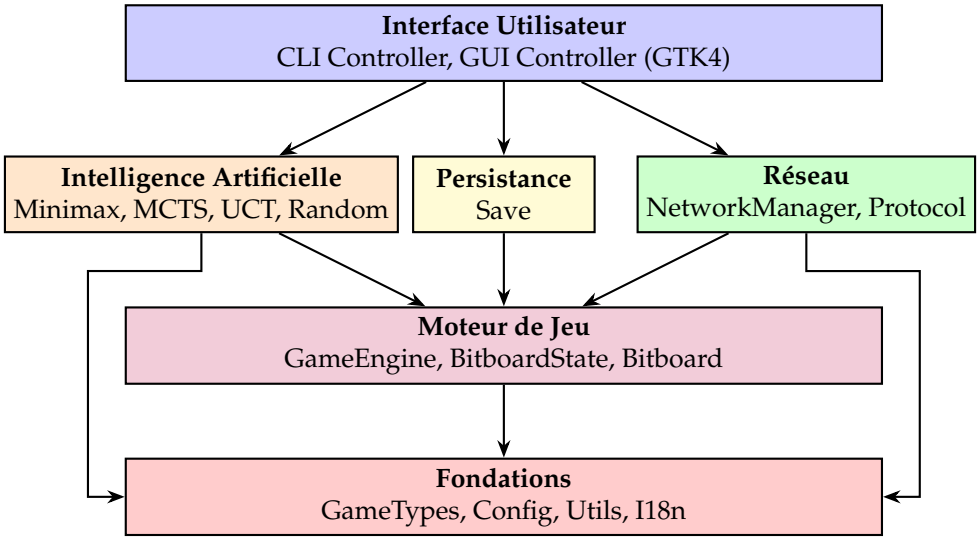


FIGURE 2 – Architecture en couches du projet. Les flèches représentent les dépendances : chaque module dépend uniquement des couches inférieures, garantissant testabilité et maintenabilité.

Le moteur de jeu constitue le **noyau pur** de l'application : totalement indépendant de l'affichage et du réseau, il expose une API pour la manipulation de l'état du jeu. Les modules d'IA interrogent ce moteur via l'interface `BitboardState` pour des calculs haute performance, tandis que les contrôleurs manipulent le `GameState`. Le réseau et l'IA communiquent avec l'interface via un système de **callbacks**, évitant ainsi toute dépendance cyclique.

3.2 Résumé de l'architecture

Le jeu des Amazones représenté en figure [**?**] est organisé en trois couches distinctes.

Interface utilisateur Deux modes d'interaction sont disponibles via l'interface `Controller`. Le `CLI_Controller` offre une interface en ligne de commande utilisant la bibliothèque `readline`, tandis que le `GUI_Controller` propose une interface graphique basée sur GTK4. Ces deux contrôleurs partagent les mêmes dépendances vers les services de la couche inférieure.

Services La couche services regroupe la logique métier en trois modules. L'`AI_Interface` constitue une interface polymorphe pour les algorithmes d'IA, définissant la méthode `get_best_move()`. Le `GameEngine` gère les règles du jeu, applique les coups et détecte la fin de partie. Enfin, le `NetworkManager` assure la gestion du multijoueur via TCP/IP avec découverte automatique des serveurs sur le réseau local.

Données La couche données contient les structures fondamentales du système. `GameState` encapsule l'état complet de la partie, incluant le plateau, le joueur actuel et les piles undo/redo. `Move` représente un coup complet avec la position source, la destination et la position de la flèche. `Bitboard` offre une représentation binaire compacte sur 128 bits permettant des calculs rapides, et `BitboardState` agrège trois bitboards distincts pour les reines blanches, les reines noires et les flèches.

Patrons de conception L'architecture exploite plusieurs patrons de conception classiques. Le patron *Strategy* permet de substituer les algorithmes d'IA de manière transparente. Le patron *Factory* délègue la création des IA à un module dédié. Le patron *Observer* gère les événements réseau de manière asynchrone via des callbacks. Le patron *Façade* simplifie l'accès au `GameEngine` en masquant la complexité des opérations sur les bitboards.

Cette architecture modulaire facilite les tests, la maintenance et l'évolution du projet.

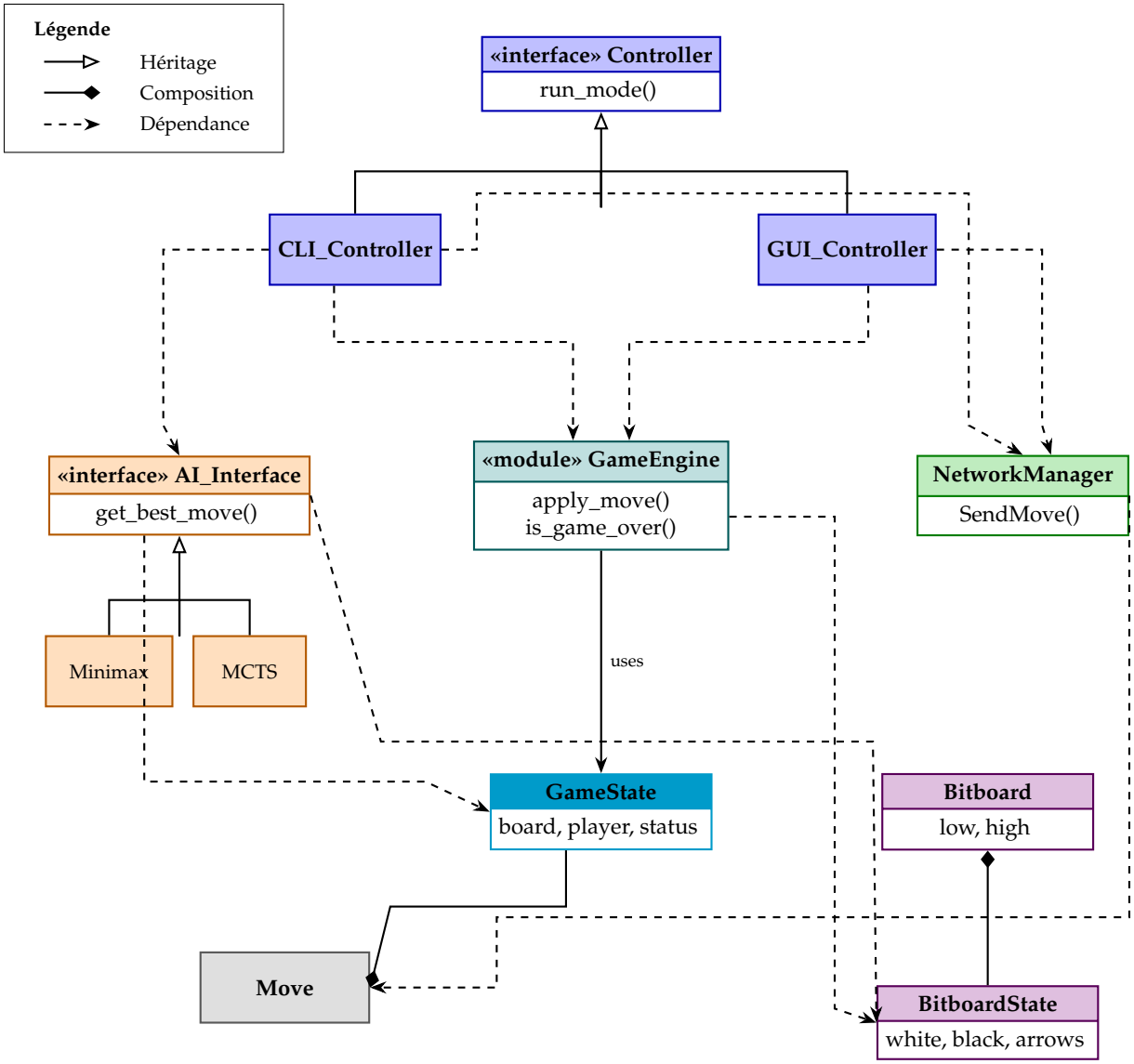


FIGURE 3 – Diagramme UML de classes simplifié. Les contrôleurs (CLI/GUI) utilisent les services (AI, GameEngine, Network). L'IA et le moteur de jeu dépendent de GameState. Le moteur utilise Bitboard et BitboardState pour les calculs haute performance.

3.3 Architecture détaillée du module IA

3.3.1 Isolation totale et interface unifiée

Le module IA constitue un composant **hermétiquement isolé** du reste de l'application. Cette séparation stricte présente trois avantages architecturaux majeurs :

— **Développement parallèle** : Durant le projet, les développeurs des interfaces (CLI/GUI) et du réseau n'ont jamais eu besoin de connaître les détails internes de Minimax ou MCTS. Seule l'interface publique `AI_Interface` était nécessaire.

— **Testabilité** : Le module IA peut être compilé et testé indépendamment via un simple programme CLI sans dépendances vers GTK4 ou le réseau.

— **Évolutivité** : L'ajout d'un nouvel algorithme (par exemple, un solveur de preuve) ne nécessite aucune modification des couches supérieures.

Cette isolation est garantie par une **interface polymorphe** (table de fonctions virtuelles ou *VTable*) qui masque complètement les détails d'implémentation. La figure 4 illustre cette architecture.

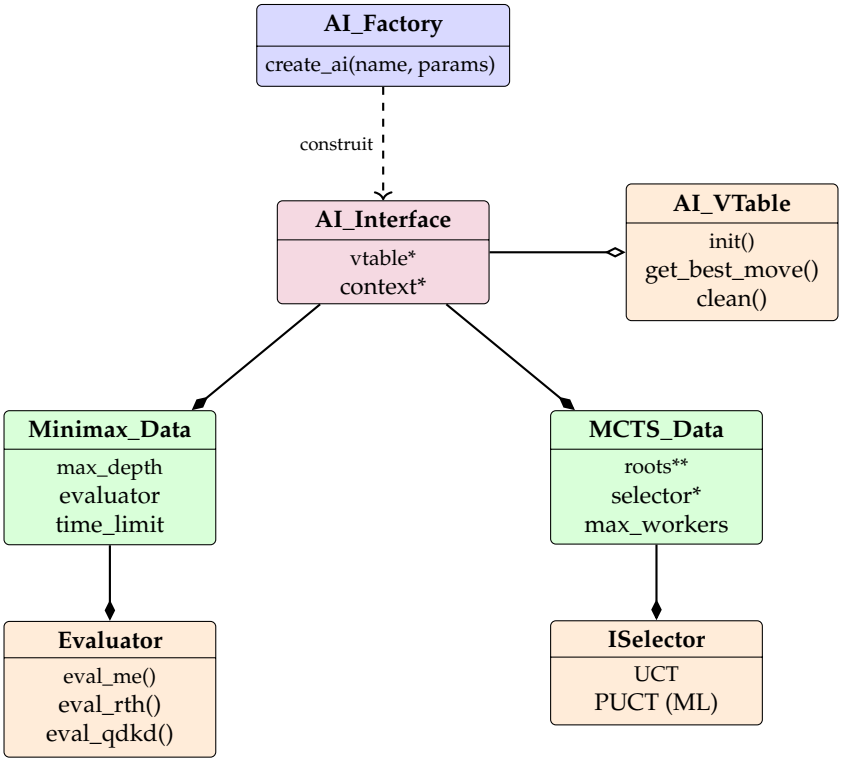


FIGURE 4 – Architecture du module IA. La couche publique expose uniquement `AI_Interface` et `AI_Factory`. Les implémentations privées sont totalement opaques pour les appelants.

3.3.2 Le patron Factory : découplage par chaîne de caractères

Le cœur de cette architecture repose sur le patron *Factory*, qui permet de créer des instances d'IA à partir de simples chaînes de caractères. L'usine maintient un registre interne associant un nom (ex : `"mcts"`) à un constructeur. Ce mécanisme élimine tout couplage statique :

```
// Les interfaces (CLI/GUI) ne dependent que d'une seule fonction
AI_Interface ai = create_ai("mcts", configuration_dynamique);
Move best = ai.vtable->get_best_move(&ai, game_state);
ai.vtable->clean(&ai);
```

Listing 1 – Création découplée depuis l'interface publique

Cette conception présente un avantage décisif lors du développement : **un membre de l'équipe peut implémenter un nouvel algorithme sans jamais toucher au code des autres modules**. Il lui suffit de créer son fichier source, d'implémenter les fonctions de la VTable, et d'ajouter une ligne dans le registre statique de l'usine. Les interfaces supérieures continueront de l'appeler via `create_ai` sans aucune modification.

3.3.3 Injection de configuration par dictionnaire dynamique

La configuration des algorithmes pose un défi architectural : chaque IA possède des paramètres spécifiques (profondeur pour Minimax, nombre de threads pour MCTS), mais l'interface publique doit rester uniforme.

La solution retenue est un **dictionnaire clé-valeur dynamique** (`ConfigMap`). Au lieu de passer des structures en dur, l'interface envoie un tableau de chaînes de caractères (par exemple `{"depth", "5"}` ou `{"max_workers", "8"}`). Cette approche évite la prolifération de structures de configuration rigides et permet l'**extension sans rupture de compatibilité**. Ajouter un nouveau paramètre d'heuristique ne casse pas l'API publique ; les anciens appelants continuent de fonctionner avec les paramètres par défaut.

Pour garantir la sécurité des types à la compilation malgré l'usage de chaînes de caractères, nous utilisons le mécanisme C11 `_Generic`. Ce dernier dispatche automatiquement le parsing vers la bonne fonction de conversion (`int`, `bool`, etc.) selon le type de la variable cible, conservant ainsi une syntaxe extrêmement propre.

3.3.4 Récursion du patron : Evaluators et Selectors

Le principe de découplage par interface a été systématiquement appliqué aux **sous-composants** des algorithmes. Minimax et MCTS ne manipulent jamais directement leurs heuristiques ou stratégies de sélection.

Dans Minimax, l'évaluation du plateau est déléguée à un simple **pointeur de fonction** (`Evaluator`). Cette indirection permet de changer d'heuristique (ME, RTH, QDKD) à la volée durant les tests de performance, en modifiant uniquement la configuration d'entrée, sans toucher au code de la recherche alpha-beta.

De manière symétrique, MCTS utilise une sous-interface (`ISelector`) pour ses stratégies de sélection. Cette architecture récursive présente un avantage majeur : **l'ajout d'une nouvelle stratégie est totalement transparent pour l'algorithme principal**. L'intégration de notre réseau de neurones (PUCT) n'a nécessité que l'implémentation de la nouvelle VTable, avec zéro modification dans le cœur du moteur MCTS.

3.3.5 Bilan : une architecture en poupées russes

L'architecture finale suit un principe de **délégation en cascade** :

$$\texttt{CLI/GUI} \xrightarrow{\text{crée}} \texttt{AI_Factory} \xrightarrow{\text{instancie}} \texttt{AI_Interface} \xrightarrow{\text{contient}} \texttt{Minimax/MCTS} \xrightarrow{\text{utilise}} \texttt{Evaluator/Selector}$$

Chaque niveau ignore les détails du niveau inférieur. Cette séparation stricte a permis durant le développement :
— Le développement parallèle de Minimax et MCTS par deux membres différents de l'équipe.
— L'intégration tardive du machine learning (semaine 10/12) sans refonte de MCTS.
— Le changement d'heuristique par défaut sans modifier une seule ligne hors des fichiers dédiés.
— La compilation conditionnelle du support TensorFlow (`ENABLE_ML`) sans casser les autres modules.

Cette architecture démontre qu'une conception orientée interfaces, même en C procédural, peut atteindre un niveau de modularité comparable aux langages orientés objet, tout en conservant les performances du C natif.

3.3.6 Modélisation matricielle et source de vérité

Afin de garantir un moteur de jeu robuste, évolutif et capable de soutenir les calculs intensifs de l'IA, l'architecture des données s'articule autour d'un modèle hybride. L'état global et officiel de la partie est encapsulé dans une structure unifiée `GameState`. La représentation structurelle de base du plateau est stockée sous forme d'un tableau dynamique à deux dimensions (matrice de type `CellContent** board`).

Cette matrice agit comme la source de vérité absolue. Chaque cellule contient une énumération définissant son état sémantique. Cette modélisation matricielle offre une lisibilité optimale pour l'interface utilisateur et permet un accès spatial direct en temps constant $\mathcal{O}(1)$. Lorsque l'IA doit évaluer l'arbre des possibles, le moteur effectue une sérialisation dynamique de cette matrice vers les structures `Bitboard`. Cette séparation des préoccupations permet de garder un code métier clair, comme l'illustre la création de l'état racine au lancement de l'algorithme :

```
static Move mcts_get_best_move(const AI_Interface *self, GameState state) {
    MCTS_Data *data = (MCTS_Data *)self->context;

    // Generation de l'etat Bitboard racine depuis la matrice UI
    BitboardState root_state = sync_from_logic(&state);


}
```

Listing 2 – Sérialisation dynamique au lancement de la recherche

3.3.7 Indépendance du code et encapsulation de la complexité

Le moteur constitue le cœur logique de l'application, totalement séparé des couches d'affichage et de communication. Son code interne ne contient aucune instruction d'entrée/sortie standard (comme la lecture de fichiers) et ignore l'existence du réseau.

Ce moteur sert de barrière stricte entre la logique mathématique du jeu et le reste du programme. La difficulté liée à la double modélisation du plateau (la matrice classique et la synchronisation des `Bitboards`) est totalement rendue invisible pour le contrôleur qui l'utilise. Il interagit avec l'extérieur exclusivement via une API minimaliste définie dans `game_engine.h` :

```
// Le controleur ignore l'existence des Bitboards ou de la pile Undo/Redo
bool is_move_legal(const GameState *state, Move move);
void apply_move(GameState *state, Move move);
PlayerColor get_winner(const GameState *state);
```

Listing 3 – Extrait de l'API publique du moteur masquant la complexité interne

Lorsqu'un module (comme le CLI ou l'IA) appelle la fonction `is_move_legal`, le moteur s'occupe en interne de préparer les données, d'appliquer l'algorithme de lancer de rayon et de vérifier les collisions, pour finalement ne renvoyer qu'un simple booléen.

Cette isolation forte garantit une réutilisation totale du code. Le même espace mémoire peut arbitrer une partie locale via GTK4, valider des données réseau, ou être cloné pour simuler des parties virtuelles pour l'IA, sans modification de sa structure interne.

3.3.8 Programmation défensive et sécurité

En tant que noyau, le moteur reçoit des appels provenant de sources variées et potentiellement imprévisibles. Chaque coordonnée envoyée est systématiquement filtrée par `is_valid_coord` pour empêcher les accès mémoire hors limites. Les fonctions publiques commencent par une validation des pointeurs, protégeant contre l'utilisation de données non initialisées. Cette solidité garantit que même si un autre module envoie des données corrompues, le moteur maintient son intégrité sans jamais faire planter l'application.

3.3.9 Architecture hybride C/Python pour l'Apprentissage Automatique

L'intégration du réseau de neurones (PUCT) repose sur une architecture hybride séparant strictement la phase d'inférence (en C) de la phase d'entraînement (en Python). Ce pont inter-langages permet d'exploiter les performances brutes du C pour la recherche arborescente, tout en bénéficiant de l'écosystème mature de Python TensorFlow/Keras pour la modélisation mathématique.

Le cycle de vie de notre modèle s'articule autour d'un pipeline de données en trois phases :

1. **Génération des données (C) :** Le moteur C fait s'affronter des IAs (heuristique QDKD) lors de parties auto-générées (*self-play*). Les états des plateaux (les tenseurs $11 \times 11 \times 4$) et les résultats finaux des parties sont sérialisés et exportés sous forme de tableaux binaires NumPy (`.npz`).
2. **Entraînement du modèle (Python) :** Un script ingère ces données pour entraîner le réseau de neurones. Le modèle final est exporté au format standard *SavedModel*.
3. **Inférence embarquée (C) :** L'exécutable C charge ce modèle pré-entraîné en mémoire via l'`API C de TensorFlow`, permettant au MCTS d'interroger le réseau neuronal à la volée durant les parties.

Architecture du réseau et fonction de coût. Contrairement à une simple régression linéaire, notre modèle est un Perceptron Multicouche (MLP). L'entrée (notre tenseur $11 \times 11 \times 4$) est d'abord aplatie (`Flatten`), puis traverse une couche cachée de 64 neurones utilisant la fonction d'activation non linéaire `ReLU`. La couche de sortie est constituée d'un neurone unique avec une activation `Sigmoïde`, contraignant la prédiction entre 0 (défaite certaine) et 1 (victoire certaine).

```
model = keras.Sequential([
    keras.layers.Input(shape=(11, 11, 4)),
    keras.layers.Flatten(),
    keras.layers.Dense(64, activation='relu'),    # Couche cachee
    keras.layers.Dense(1, activation='sigmoid')  # Prediction binaire
])
model.compile(optimizer='adam', loss='binary_crossentropy')
```

Listing 4 – Architecture du Perceptron Multicouche (Keras)

Le modèle est optimisé par l'algorithme `Adam`, avec pour objectif de minimiser l'erreur d'entropie croisée binaire (`Binary Cross-Entropy`), une fonction de perte parfaitement adaptée à l'évaluation de probabilités pour des problèmes de classification binaire (Gagner/Perdre).

Prévention du surapprentissage (*Overfitting*). Pour garantir que l'IA développe une réelle intuition stratégique plutôt que d'apprendre bêtement les parties par cœur, nous avons implémenté une division stricte des données (80 % pour l'entraînement, 20 % pour la validation). À chaque itération (*epoch*), les performances du modèle sont évaluées sur le sous-ensemble de validation qu'il n'a jamais étudié. Le suivi de ces deux courbes nous permet de trouver le point d'équilibre optimal (*sweet spot*) et d'interrompre l'apprentissage avant le surapprentissage. Le script Python complet gérant ce pipeline est disponible en Annexe.

3.4 Architecture du module réseau

L'architecture du module réseau repose sur trois piliers conceptuels : une façade centralisée masquant la complexité des sockets, un modèle événementiel mono-threadé basé sur le multiplexage, et un système de rappels découplant le cœur réseau des interfaces utilisateur. La figure 5 illustre l'organisation générale de ces composants.

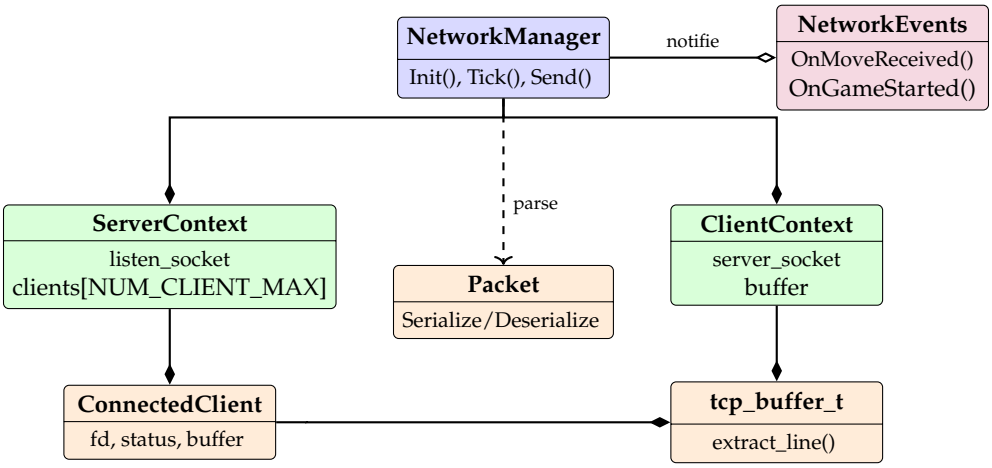


FIGURE 5 – Diagramme de composition du `NetworkManager`. La façade possède au plus un contexte serveur **ou** client (exclusion mutuelle). Chaque client embarque son propre tampon TCP. Légende : ♦ composition, ◊ agrégation, trait pointillé = dépendance.

3.4.1 La façade `NetworkManager`

Le module réseau expose une unique structure publique — `NetworkManager` — qui encapsule l'intégralité de la logique de communication. Les couches supérieures (CLI, GUI) n'ont jamais besoin de manipuler directement les descripteurs de fichiers, les structures d'adresses réseau, ni les tampons de framing. Ce masquage strict de l'information garantit que toute évolution interne (passage à TLS, changement de protocole) n'impacte pas le code client.

L'initialisation de la façade illustre cette encapsulation :

```
void NetworkManager_Init(NetworkManager *manager, NetworkEvents callbacks) {
  manager->current_mode = NET_MODE_NONE;
  manager->server_ctx = NULL;
  manager->client_ctx = NULL;
  manager->callbacks = callbacks;  // Injection des rappels
}
```

Listing 5 – Initialisation du NetworkManager

Le `NetworkManager` peut fonctionner en trois modes mutuellement exclusifs : inactif, serveur ou client. En mode serveur, il alloue un `ServerContext` contenant la socket d'écoute, le tableau des 64 slots clients avec leurs tampons TCP respectifs, le tableau des 32 parties actives, et le contexte de découverte UDP. En mode client, il alloue un `ClientContext` minimaliste contenant uniquement la socket de connexion au serveur et son tampon de réception. Cette ségrégation mémoire évite toute confusion entre les rôles et permet une libération ciblée des ressources.

3.4.2 Le multiplexage asynchrone avec `poll()`

Pour gérer simultanément jusqu'à 64 connexions sans threads, nous avons opté pour le multiplexage d'entrées/sorties via l'appel système `poll()`. Trois alternatives existaient : `select()`, limité à 1 024 descripteurs et utilisant des masques de bits peu pratiques ; `epoll()`, plus performant mais spécifique à Linux ; et `poll()`, offrant une interface portable POSIX avec des performances adaptées à notre charge. Le modèle mono-threadé élimine par construction les conditions de course et la nécessité de verrous, simplifiant considérablement le raisonnement sur la correction du code.

La fonction `NetworkManager_Tick()`, appelée à chaque itération de la boucle principale, invoque `poll()` avec un timeout nul. Ce paramétrage rend l'appel instantanément retourné : si des données sont disponibles, elles sont traitées ; sinon, le contrôle revient immédiatement à l'appelant. Cette conception permet à l'interface de continuer à traiter d'autres événements (saisie clavier, rafraîchissement graphique) entre deux cycles réseau.

```
static void handle_server_tick(NetworkManager *manager, ServerContext *ctx) {
  int poll_count = poll(ctx->fds, MAX_CLIENTS + 1, POLL_TIMEOUT_IMMEDIATE);
```

```
3
4   if (poll_count > 0) {
5     if (ctx->fds[LISTEN_SOCKET_INDEX].revents & POLLIN)
6       handle_server_new_connection(ctx);
7
8     for (int i = FIRST_CLIENT_INDEX; i <= MAX_CLIENTS; i++) {
9       if (ctx->fds[i].fd != INVALID_SOCKET &&
10          (ctx->fds[i].revents & (POLLIN | POLLHUP | POLLERR)))
11        handle_server_client_data(manager, ctx, i);
12    }
13  }
14 }
```

Listing 6 – Tick serveur avec poll() non bloquant

La boucle principale du CLI illustre une application avancée de ce modèle : elle doit surveiller simultanément deux sources d'événements indépendantes — le réseau et le clavier. Plutôt que de créer deux threads ou de bloquer sur l'une des sources, nous utilisons deux appels `poll()` distincts à chaque itération : le premier interroge les sockets réseau avec un timeout nul (traitement immédiat des paquets entrants), le second interroge `stdin` avec un court timeout permettant la saisie de commandes. Cette architecture permet au joueur de taper des commandes tout en recevant instantanément les coups de l'adversaire, simulant un parallélisme sans les risques inhérents à la programmation concurrente.

3.4.3 Le patron Observateur pour le découplage

La communication entre le module réseau et les couches supérieures est régie par le patron *Observer*. À l'initialisation, le `NetworkManager` reçoit une structure `NetworkEvents` contenant neuf pointeurs de fonctions couvrant tous les événements significatifs : réception d'un coup, connexion ou déconnexion d'un joueur, réception d'une invitation, démarrage d'une partie, erreur protocolaire, expiration du temps de réflexion, et réception d'une liste de joueurs.

Lorsque le réseau reçoit un coup valide, il se contente d'invoquer le rappel `OnMoveReceived(move)` sans connaître l'identité de l'appelant. C'est la couche d'interface qui décide de la réaction appropriée : le CLI met à jour l'affichage textuel du plateau et réaffiche le prompt, tandis que la GUI redessine les cases concernées via les primitives GTK. Cette inversion de contrôle garantit que le module réseau reste totalement indépendant des interfaces utilisateur : les fichiers d'implémentation CLI et GUI fournissent chacun leur propre jeu de rappels sans qu'une seule ligne du cœur réseau n'ait été modifiée.

3.4.4 Routage des paquets et arbitrage serveur

Une fois un paquet désérialisé, il est acheminé vers un gestionnaire dédié via une structure de sélection indexée par le type de paquet. Cette conception respecte le principe ouvert/fermé : ajouter un nouveau type de message nécessite uniquement l'ajout d'un cas dans le gestionnaire, sans modifier la logique de routage existante.

```
1  void handle_server_packet(NetworkManager *manager, ServerContext *ctx,
2                            int client_index, Packet *pkt) {
3    switch (pkt->type) {
4      case C2S_JOIN:   process_join_packet(manager, ctx, client_index, pkt->payload); break;
5      case C2S_INVITE: process_invite_packet(manager, ctx, client_index, pkt->payload); break;
6      case C2S_MOVE:   process_move_packet(manager, ctx, client_index, pkt->payload); break;
7      case C2S_HEARTBEAT: break;  // Mise a jour silencieuse du timestamp
8      // ... autres cas
9      default: printf("[SERVER] Unmanaged packet (Type: %d)\n", pkt->type);
10   }
11 }
```

Listing 7 – Routage des paquets par type

Le serveur ne se contente pas de relayer aveuglément les coups entre joueurs. Avant tout transfert, il maintient une copie locale du plateau pour chaque partie active et valide la légalité de chaque coup reçu. Si un coup est déterminé illégal par le moteur de jeu, il est rejeté avec un message d'erreur explicite envoyé au client fautif, et l'adversaire n'est jamais notifié de cette tentative. Ce mécanisme garantit qu'un client malveillant ou bogué ne peut pas corrompre l'état de jeu de son adversaire ni obtenir un avantage indu.

```
1  // Extrait de process_move_packet()
2  GameState *server_board = &ctx->active_games[game_idx];
3
4  if (is_move_legal(server_board, parsed_move)) {
5    apply_move(server_board, parsed_move);  // MAJ plateau arbitre
```

```
  // Relais vers l'adversaire...
} else {
  printf("[ANTI-CHEAT] Client %d tried illegal move\n", client_index);
  send_error(manager, client_index, "Illegal move detected!");
}
```

Listing 8 – Validation anti-triche des coups

Chaque partie active possède un identifiant unique attribué séquentiellement par le serveur. Lorsqu'un client envoie un coup, le serveur consulte son champ `game_id` pour identifier la partie concernée, puis parcourt les clients connectés pour trouver celui partageant le même identifiant avec un statut « en jeu ». Cette recherche, bien que linéaire dans le pire cas ($\mathcal{O}(n)$ où n est le nombre de clients), reste négligeable en pratique car le nombre de clients actifs est borné.

3.4.5 Structures de contexte et gestion des connexions

Chaque client est représenté par une structure à taille fixe regroupant l'ensemble des informations nécessaires à sa gestion : le descripteur de socket, son tampon TCP dédié, son pseudo, son statut courant dans la machine à états, l'identifiant de sa partie en cours, l'horodatage de sa dernière activité, et les métadonnées d'invitation pendante. Cette encapsulation permet de traiter chaque client comme une entité autonome sans variables globales partagées.

Une décision architecturale majeure concerne l'allocation mémoire : les 64 slots clients et les 32 slots de parties sont alloués **une seule fois** au démarrage du serveur. Aucune allocation dynamique n'intervient lors des connexions ou déconnexions : un client qui se connecte occupe un slot précédemment marqué invalide, et sa déconnexion remet simplement ce marqueur. Ce patron de *pool d'objets* présente trois avantages : il élimine la fragmentation mémoire sur les serveurs à longue durée de vie, il garantit un temps de traitement constant par connexion ($\mathcal{O}(1)$ pour trouver un slot libre dans le pire cas via parcours linéaire borné), et il rend l'empreinte mémoire totale déterministe et prévisible dès le démarrage.

Une optimisation critique réside dans l'alignement des tableaux de surveillance et de métadonnées : l'index i dans le tableau `fds[]` utilisé par `poll()` correspond **exactement** à l'index i dans le tableau `clients[]`. Cette correspondance bijective permet de passer en $\mathcal{O}(1)$ du descripteur de fichier signalé par le système d'exploitation aux informations complètes du joueur (pseudo, statut, partie en cours), sans recourir à une recherche linéaire ou à une table de hachage. L'index 0 est réservé à la socket d'écoute du serveur, les index 1 à 64 correspondent aux clients connectés.

3.4.6 Sérialisation et framing TCP

Chaque paquet suit le format textuel `"TYPE|PAYLOAD\n"`. La sérialisation et désérialisation s'effectuent en temps linéaire par rapport à la taille du message :

```
int SerializePacket(const Packet *pkt, char *buffer_out, size_t max_len) {
  return snprintf(buffer_out, max_len, "%d|%s\n", pkt->type, pkt->payload);
}
```

Listing 9 – Sérialisation des paquets

Le tampon TCP accumule les fragments jusqu'à réception d'un délimiteur `'\n'`. L'extraction utilise `memchr()` pour la recherche et `memmove()` pour la compaction :

```
int tcp_buffer_extract_line(tcp_buffer_t *buf, char *out_msg, size_t max_len) {
  char *newline = memchr(buf->data, '\n', buf->length);
  if (newline == NULL) return EXTRACT_NO_MSG;

  size_t msg_len = newline - buf->data;
  memcpy(out_msg, buf->data, msg_len);
  out_msg[msg_len] = '\0';

  size_t remaining = buf->length - msg_len - 1;
  memmove(buf->data, newline + 1, remaining);  // Compaction
  buf->length = remaining;
  return EXTRACT_SUCCESS;
}
```

Listing 10 – Extraction d'une ligne du tampon TCP

3.5 Le module d'interface graphique (GUI)

L'architecture du module graphique, mise en place avec la bibliothèque GTK4 [17], a été conçue selon le pattern **Modèle-Vue-Contrôleur (MVC)**. L'enjeu principal de ce module consistait à interfacer une boucle d'événements purement asynchrone avec un moteur de jeu déterministe et des services potentiellement bloquants (Réseau, Intelligence Artificielle). La figure 6 présente l'architecture autour de la GUI.

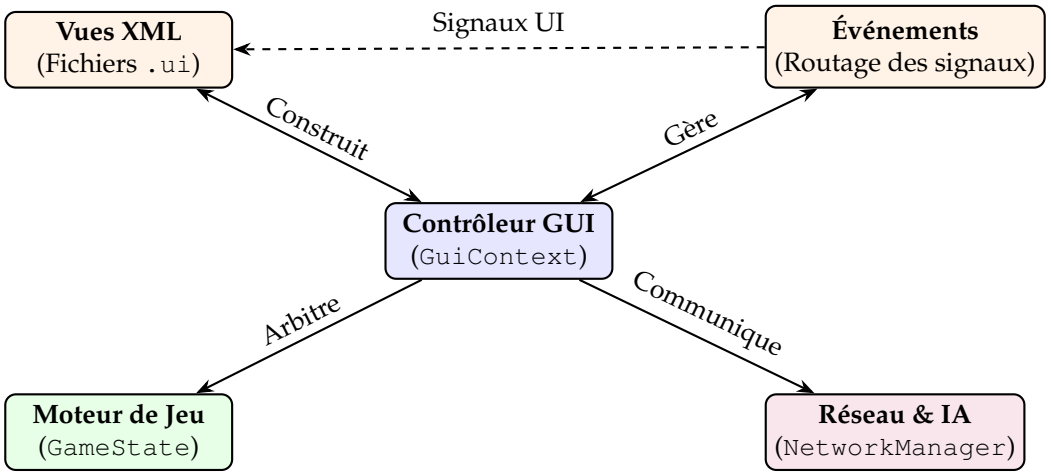


FIGURE 6 – Architecture du module graphique : le contrôleur orchestre les vues asynchrones, le réseau et le modèle de jeu.

3.5.1 Séparation déclarative et encapsulation des ressources

L'interface visuelle est dissociée du code C via des fichiers de description déclarative au format XML (fichiers `.ui`). Cette séparation présente un double avantage : elle permet la modification de la topologie de l'interface sans recompilation de la logique applicative, et elle concentre l'initialisation du code sur l'injection de dépendances (*event binding*) via `GtkBuilder`.

Pour garantir la portabilité de l'exécutable, ces ressources XML (ainsi que les feuilles de style CSS) sont compilées nativement dans le binaire final via le système `GResource`. L'application est ainsi livrée sous forme d'un exécutable autonome (*standalone*), éliminant les risques liés aux chemins de fichiers relatifs manquants sur la machine cible.

3.5.2 L'orchestrateur asynchrone et la Main Loop

Le défi majeur d'une interface graphique développée en C est le syndrome de l'interface figée (*UI freeze*). Le système de fenêtrage repose sur une boucle principale (*Main Loop*) qui ne doit jamais être bloquée.

Pour y parvenir, le contrôleur `GuiContext` agit comme un **orchestrateur asynchrone**. Les opérations légères mais récurrentes (comme le `Tick()` du gestionnaire réseau) sont greffées sur la boucle GTK via des *timeouts* (`g_timeout_add`). À l'inverse, les calculs lourds d'IAs sont instanciés dans des *threads* POSIX parallèles (`g_thread_new`). Pour garantir la *thread-safety* absolue lors du rafraîchissement visuel, le thread d'IA utilise la primitive `g_idle_add` pour réinjecter de manière sécurisée son résultat (le meilleur coup calculé) dans le thread principal de GTK. De plus, des conditions bloquent totalement certaines actions qui pourraient corrompre la partie en cours si le thread de l'IA est actuellement actif. L'interface reste ainsi fluide et réactive en toutes circonstances.

3.5.3 Automate fini interactif et approche transactionnelle

Contrairement aux échecs, un tour aux Amazones est une transaction en deux temps (déplacement, puis tir). Pour empêcher toute corruption de l'état du jeu par des clics erratiques, les interactions avec le plateau (clics et *Drag & Drop* natif via `GtkDragSource` / `GtkDropTarget`) sont régies par une **machine à états finis (FSM)** stricte.

Le système oscille entre `PHASE_MOVE_QUEEN` et `PHASE_SHOOT_ARROW`. Lors du déplacement de la reine, l'interface adopte une approche **transactionnelle retardée** : le déplacement n'est opéré qu'en mémoire graphique locale pour fournir un retour visuel immédiat à l'utilisateur. La méthode de mutation officielle de l'état du noyau (`apply_move`) n'est invoquée qu'à l'issue complète de la transaction (validation du tir de la flèche). Ainsi, le moteur central et sa pile d'historique (Undo/Redo) ne reçoivent jamais d'états partiels ou corrompus. En cas d'annulation par clic droit durant la phase de tir, la transaction graphique est simplement avortée (*rollback* visuel).

3.5.4 Navigation lâche et stylisation dynamique

La gestion de la navigation entre les différents écrans (Menu, Jeu, Lobby, Configuration) s'articule autour d'un conteneur `GtkStack`. Le contrôleur utilise un couplage lâche : plutôt que de coder en dur les transitions, les bou-

tons de l'interface intègrent l'identifiant de leur page cible directement dans leurs balises XML personnalisées. Le contrôleur se contente de router génériquement ces requêtes.

Enfin, la clarté visuelle du plateau de jeu est assurée par un `GtkCssProvider`. Au lieu de manipuler pixel par pixel la couleur des cases, le contrôleur applique et retire dynamiquement des classes CSS sémantiques (`.piece-white`, `.highlight-valid`, `.hint-arrow`). Cette approche permet non seulement de thématiser l'application en un instant, mais délègue surtout le moteur de rendu au processeur graphique (GPU) interne de GTK4, assurant des animations et des mises en surbrillance d'une grande fluidité.

3.6 Architecture du module CLI

Le module CLI a été conçu selon des principes stricts de séparation des préoccupations, garantissant que la logique d'interface n'interfère jamais avec les règles du moteur de jeu.

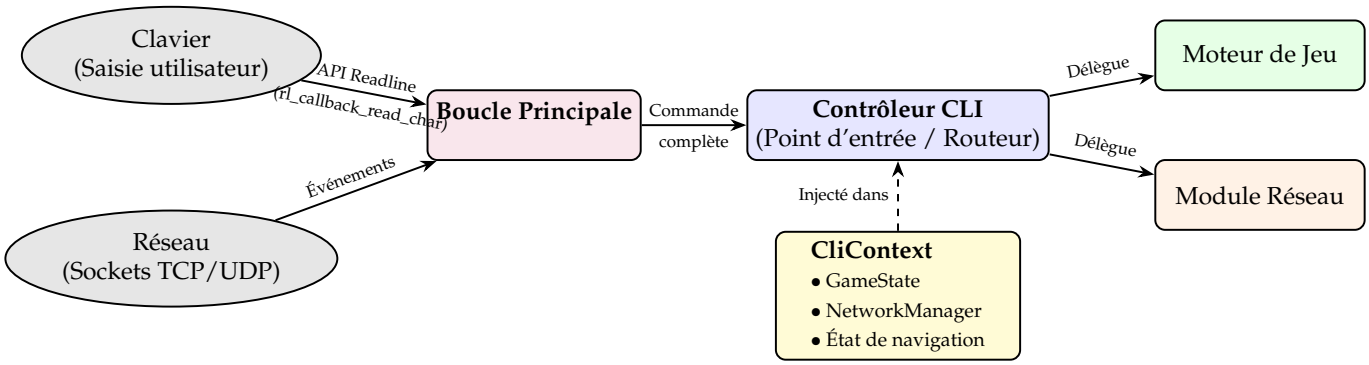


FIGURE 7 – Architecture du module CLI : intégration asynchrone des entrées, routage centralisé et encapsulation de l'état d'exécution.

3.6.1 Le contexte CLI (`CliContext`) et le routage centralisé

Pour éviter les variables globales, l'ensemble de l'état de la session est centralisé dans une structure `CliContext` transmise à chaque fonction. Cette structure agit comme un patron de conception *State* : elle regroupe le pointeur vers la partie en cours, l'état de navigation (menu, lobby réseau, en jeu), les instances d'IA et le gestionnaire réseau.

L'analyse des commandes fonctionne sur le principe d'un routeur centralisé. L'algorithme isole le premier mot saisi et le compare à un dictionnaire de commandes via des constantes symboliques. L'exécution finale dépend de la validation de l'état présent dans le `CliContext`. Le contrôleur peut ainsi bloquer des commandes illogiques (ex : empêcher `undo` en partie multijoueur) sans jamais solliciter le moteur de jeu.

3.6.2 Boucle événementielle hybride et multiplexage

Le contrôleur CLI implémente une boucle principale non bloquante devant surveiller simultanément le clavier et le réseau. Plutôt que de recourir au multi-threading complexe, nous utilisons un double multiplexage avec `poll()`.

À chaque itération, un premier `poll()` réseau avec un timeout nul traite instantanément les paquets entrants. Ensuite, la logique de jeu (fin de partie, tour IA, timeout) est évaluée. Enfin, un second `poll()` interroge l'entrée standard (`stdin`) avec un timeout court. Cette architecture permet au joueur de saisir des commandes tout en recevant les coups de l'adversaire en temps réel, simulant un parallélisme robuste.

3.6.3 Sérialisation et robustesse du parsing

La persistance des données (sauvegarde/chargement) repose sur une analyse syntaxique séquentielle en temps linéaire $\mathcal{O}(N)$. Avant l'analyse, chaque ligne subit un pré-traitement strict filtrant les commentaires (`{` ou `#`) et les espaces redondants, protégeant l'application contre les fichiers au formatage capricieux.

L'analyseur maintient une variable d'état pour s'orienter entre les sections (paramètres, plateau, historique). Cette conception impose une sécurité d'ordre logique : la dimension du plateau doit impérativement être validée dans les paramètres avant d'allouer la matrice ou de reconstruire l'historique, empêchant toute corruption mémoire.

3.6.4 Validation mathématique et sécurité des entrées

La conversion de la notation algébrique standard (ex : `d4`) en coordonnées cartésiennes (x, y) constitue une ultime barrière de sécurité. Pour un plateau $D \times D$, la conversion s'effectue par des soustractions sur l'ordre alphabétique :

$$x = \text{lettre} - \text{'a'} \qquad y = \text{chiffre} - 1$$

Une validation stricte des bornes $(0 \leq x < D) \wedge (0 \leq y < D)$ est appliquée immédiatement. Cette vérification mathématique empêche les dépassements de capacité inhérents aux variables non signées en cas de résultat négatif. Le moteur de jeu ne reçoit ainsi que des instructions préalablement assainies.

4 Explication approfondie

4.1 Fonctionnalité réseau : Système d'invitations et machine à états (F40)

4.1.1 Prérequis et problème

Le cahier des charges (F40) impose un système complet d'invitations permettant aux joueurs de se défier dans le lobby. Ce système doit gérer plusieurs cas complexes : un joueur peut être indisponible (`AWAY`), déjà en partie (`INGAME`), ou en attente de réponse (`WAITGAME`). Les invitations doivent expirer automatiquement après 5 minutes, et toute transition d'état invalide doit être rejetée avec un message explicite.

4.1.2 Solution implémentée : machine à états finis

Nous avons modélisé le cycle de vie d'un client par un automate fini à cinq états. Chaque transition est gardée par des vérifications explicites :

```
typedef enum {
  CLIENT_STATUS_CONNECTED,  // TCP etabli, attente JOIN
  CLIENT_STATUS_IDLE,       // Dans le lobby, disponible
  CLIENT_STATUS_AWAY,       // Indisponible temporairement
  CLIENT_STATUS_WAITGAME,   // Invitation en attente
  CLIENT_STATUS_INGAME      // Partie en cours
} ClientStatus;
```

Listing 11 – Énumération des statuts clients

Traitement d'une invitation. Lorsqu'un joueur envoie une invitation, le serveur vérifie l'éligibilité via des gardes d'état :

```
// Extrait de process_invite_packet()
if (ctx->clients[sender_index].status != CLIENT_STATUS_IDLE) {
  send_error(manager, sender_index, "You must be IDLE to invite.");
  return;
}
if (ctx->clients[target_id].status == CLIENT_STATUS_AWAY) {
  send_error(manager, sender_index, "Player is currently AWAY.");
  return;
}
// Transitions d'etat apres validation
ctx->clients[sender_index].status = CLIENT_STATUS_WAITGAME;
ctx->clients[target_id].pending_invite_from = sender_index;
```

Listing 12 – Gardes de transition pour les invitations

Expiration automatique. À chaque tick serveur, les invitations pendantes sont vérifiées. Une invitation non répondue après 300 secondes déclenche le retour au statut `IDLE` pour les deux joueurs avec notification explicite.

4.1.3 Garanties de cohérence

Cette implémentation garantit trois propriétés : **déterminisme** (chaque paquet produit exactement une transition ou un rejet), **isolation** (un joueur en partie ne peut recevoir d'invitations), et **récupération automatique** (les timeouts libèrent les ressources bloquées).

4.2 Fonctionnalité 2 : Approfondissement itératif avec contrainte de temps

4.2.1 Prérequis et problème

L'algorithme Minimax classique explore l'arbre de jeu jusqu'à une profondeur fixe d, puis retourne le meilleur coup trouvé. Cette approche présente un défaut majeur : si d est trop grand, le calcul peut dépasser le temps imparti, ne retournant aucun coup valide. Si d est trop petit, l'IA joue de manière myope. Le cahier des charges (F34) impose donc un mécanisme garantissant le retour du **meilleur coup trouvé à la profondeur maximale complétée** avant l'expiration du délai.

4.2.2 Solution implémentée : Iterative Deepening

L'approfondissement itératif consiste à explorer l'arbre en profondeur croissante ($d = 1, 2, 3, \ldots$) jusqu'à épuisement du temps. À chaque itération complète, le meilleur coup est mis à jour. Si le temps expire en milieu d'exploration, le coup de l'itération précédente est retourné.

Implémentation. La fonction `minimax_get_best_move` contient une boucle principale :

```
Move best_move_overall = available_moves[0];

for (int d = 1; d <= data->max_depth; d++) {
    if (get_time_ms() - start_time >= data->time_limit)
        break;

    // Tri des coups pour placer le meilleur de l'iteration
    // precedente en tete (ameliore le pruning)
    if (d > 1)
        sort_moves(count, best_move_overall, available_moves);

    Move move_at_depth = run_root_search(data, &root_state, d,
                                         start_time, available_moves, count);

    // Sauvegarde uniquement si l'iteration s'est terminee
    if (get_time_ms() - start_time < data->time_limit) {
        best_move_overall = move_at_depth;
    }
}

return best_move_overall;
```

Listing 13 – Boucle d'approfondissement itératif

Optimisation par réordonnancement des coups. Avant chaque nouvelle profondeur, la fonction `sort_moves` place le meilleur coup de l'itération précédente en tête du tableau. Cette heuristique exploite le fait que le coup optimal à profondeur $d-1$ est souvent proche de celui à profondeur d, ce qui améliore significativement l'efficacité de l'élagage alpha-beta en explorant d'abord les branches prometteuses.

Garanties temporelles. La vérification `get_time_ms() - start_time < data->time_limit` est effectuée :

— Avant chaque nouvelle itération de profondeur
— À l'intérieur de `alpha_beta_forward` avant chaque expansion de nœud
— Avant la sauvegarde du meilleur coup

Cette triple vérification garantit que l'algorithme ne dépasse jamais le temps alloué de plus de quelques millisecondes (le temps d'une seule expansion de nœud).

4.2.3 Intégration avec Forward Pruning (F33)

Lorsque le forward pruning est activé, chaque itération de profondeur bénéficie d'un faisceau de recherche adaptatif. La largeur du faisceau croît avec la profondeur restante :

```
int get_beam_width(int maximizing, int current_depth, int max_depth) {
    int base = maximizing ? NB_BEST_MOVES_PLAYER : NB_BEST_MOVES_OPPONENT;
    int depth_bonus = (max_depth - current_depth) * 3;
    return base + depth_bonus;
}
```

Ainsi, à profondeur 1, seuls 10 coups sont explorés (joueur maximisant), mais à profondeur 5 avec $d_{\max} = 5$, le faisceau s'élargit à $10 + (5-1) \times 3 = 22$ coups. Cette approche équilibre vitesse (petits faisceaux en profondeur) et précision (faisceaux larges près de la racine).

5 Stratégie de test et assurance qualité

La fiabilité d'un projet de cette envergure ne peut reposer uniquement sur des vérifications manuelles. Nous avons mis en place une chaîne complète d'assurance qualité intégrant tests unitaires, couverture de code, analyse statique, vérification de style et documentation embarquée. Cette section détaille chacun de ces piliers.

5.1 Framework de test : GoogleTest

L'ensemble des tests repose sur le framework **GoogleTest** (v1.15.2), téléchargé et compilé automatiquement par CMake via `FetchContent`. Bien que notre code source soit intégralement écrit en C, GoogleTest est un framework C++ : chaque fichier de test inclut donc les en-têtes C au sein de blocs `extern "C" {}` pour assurer une interopérabilité transparente entre les deux langages. Ce choix nous offre l'expressivité des macros `TEST`, `TEST_F`, `EXPECT_EQ`, `EXPECT_TRUE`, etc., tout en testant du code C pur sans modification.

Au total, le projet comporte **plus de 600 cas de test** répartis dans **25 fichiers**, couvrant l'ensemble des modules de l'application.

5.2 Infrastructure de test : fixtures et mocks

Pour les modules dont l'initialisation requiert un état complexe, nous avons développé des **fixtures de test** (classes héritant de `::testing::Test`) qui automatisent la mise en place et le nettoyage de l'environnement.

Fixture réseau (`NetworkManagerTest`). Cette fixture instancie un `NetworkManager` serveur et un client sur un port de test incrémenté automatiquement, connecte des callbacks de comptage (`mock_on_player_joined`, `mock_on_error`, etc.), et libère proprement les sockets dans son `TearDown()`. Elle permet de tester le cycle complet d'une connexion — du *handshake* TCP jusqu'au routage des coups — sans dépendance à un réseau externe.

Fixture contrôleur (`ControllerTestFixture`). Elle pré-initialise un `CliContext` complet avec un `GameState`, une configuration, un `NetworkManager` et les événements réseau associés. Chaque test hérite ainsi d'un environnement CLI fonctionnel, isolé des effets de bord grâce au nettoyage systématique du contexte global dans le `TearDown()`.

Fixture sauvegarde (`SaveLoadTest`). Elle alloue dynamiquement un plateau 10×10 avec une configuration initiale connue, permettant de tester les cycles complets de sérialisation et désérialisation sans fichier externe.

Interception de `malloc` pour les tests de robustesse. Les tests du module Minimax interceptent l'appel système `malloc` via `dlsym(RTLD_NEXT, ...)` pour simuler des échecs d'allocation mémoire. Un garde RAII MallocFailGuard active et désactive l'interception de manière sûre, même en cas d'exception, garantissant que l'IA réagit correctement face à des conditions de mémoire dégradées.

5.3 Couverture de code avec `gcovr`

La couverture de code est instrumentée via le drapeau `-coverage` de GCC, activé par un type de build CMake dédié (`-DCMAKE_BUILD_TYPE=Coverage`). Après exécution de la suite de tests, le script `gcovr` génère un rapport HTML interactif permettant de visualiser, fichier par fichier et ligne par ligne, les portions de code effectivement traversées.

La procédure complète est la suivante :

```
mkdir coverage && cd coverage
cmake -DCMAKE_BUILD_TYPE=Coverage ..
make
make test
make coverage       # genere coverage/index.html
```

Listing 14 – Génération du rapport de couverture

Ce rapport nous a permis d'identifier et de combler les zones non couvertes, notamment les cas aux limites des fonctions de validation de coups et les chemins d'erreur du parseur de sauvegarde. Les modules les plus critiques (moteur de jeu, bitboards, IA) atteignent un taux de couverture satisfaisant, tandis que les modules d'interface (GUI notamment) sont plus difficiles à tester de manière automatisée en raison de leur dépendance à GTK.

5.4 Analyse statique avec `clang-tidy`

L'analyse statique du code source est réalisée par `clang-tidy`, configuré via un fichier `.clang-tidy` à la racine du projet. Nous avons activé l'ensemble des vérifications du module `clang-analyzer-*`, qui détecte les déréférencements de pointeurs nuls, les fuites mémoire, les variables non initialisées et les divisions par zéro potentielles. L'outil est invoqué sur l'ensemble des fichiers sources :

```
clang-tidy $(find src include -name '*.[ch]') -- -std=c11 -Iinclude
```

Listing 15 – Exécution de l'analyse statique

Les avertissements jugés non pertinents dans un contexte donné sont supprimés localement via le commentaire `// NOLINT(...)`, garantissant un bilan d'analyse propre sans masquer les véritables problèmes.

5.5 Conformité de style avec `clang-format`

La cohérence du style de code à travers les contributions des cinq membres de l'équipe est assurée par `clang-format`, configuré avec le style de base **GNU** via un fichier `.clang-format` à la racine du projet. La commande suivante reformate automatiquement l'ensemble du code :

```
clang-format -i $(find src include -name '*.[ch]')
```

Listing 16 – Formatage automatique du code

Ce fichier de configuration, versionné avec le code source, garantit que tous les développeurs appliquent les mêmes conventions d'indentation, d'espacement et de placement des accolades, indépendamment de leur éditeur de texte.

5.6 Tentative de fuzzing

Nous avons exploré le *fuzzing* comme technique complémentaire de découverte de bogues, l'objectif étant de soumettre des entrées aléatoires ou semi-aléatoires aux fonctions de parsing (désérialisation des paquets réseau, parseur de fichiers de sauvegarde) afin de détecter des crashs ou des comportements indéfinis. Cependant, malgré plusieurs tentatives de mise en place, nous n'avons pas réussi à exploiter les résultats de manière concluante : l'interprétation des sorties du *fuzzer* s'est avérée difficile et nous n'avons pas pu en déduire d'anomalies actionnables dans le temps imparti. Cette piste reste néanmoins une amélioration pertinente pour renforcer la robustesse du parseur réseau et du module de sauvegarde.

6 Revue critique du projet

6.1 Ce qui a été accompli : une architecture synergique et robuste

L'intégralité du cahier des charges a été respectée, avec un taux d'achèvement de 98 % sur les composants les plus complexes comme le réseau. Plutôt que de développer des modules isolés, notre réussite repose sur une architecture où chaque couche s'appuie sur les garanties de la couche inférieure, de la logique interne jusqu'à l'expérience utilisateur.

6.1.1 Un socle logique performant au service de l'IA

Le cœur de notre application, le moteur de jeu, a parfaitement rempli ses objectifs de stabilité. Pour nous prémunir contre les fuites et la fragmentation, nous avons privilégié une architecture à empreinte mémoire constante, reposant sur des allocations statiques strictement dimensionnées pour l'état et l'historique de la partie. Ce choix structurel garantit un comportement prévisible, même lors de longues sessions.

Sur le plan des performances, le compromis entre une matrice classique (pour la lisibilité de la logique) et les *Bitboards* (pour la vitesse d'exécution) s'est avéré redoutablement efficace. Couplé à un algorithme de *Raycasting* validant les trajectoires sans latence, le moteur est capable de générer les états à très haute fréquence. Cette fluidité bénéficie directement au module IA : elle permet à notre algorithme MCTS, soutenu par l'accélération matérielle OpenMP, d'explorer l'arbre des possibles avec une profondeur et une rapidité garantissant un excellent niveau de jeu (20000 itérations par seconde, auquel il est possible d'ajouter des threads). De plus, l'isolation de ce module a rendu possible la mise en concurrence de différentes heuristiques de manière totalement transparente.

6.1.2 Résilience, sécurité et intégrité multijoueur

Cette philosophie de rigueur s'applique à l'ensemble de nos interfaces, avec pour principe fondamental de systématiquement valider les données entrantes. Cette sécurisation se décline à plusieurs niveaux :

— **Étanchéité des contextes de jeu :** La machine à états de l'interface bloque toute action illogique, comme tenter un déplacement local alors que l'utilisateur se trouve dans un lobby réseau. Cela garantit l'intégrité absolue de la matrice de jeu.

— **Serveur événementiel optimisé :** Le module réseau repose sur une architecture mono-threadée utilisant l'appel système `poll()`. Il peut gérer jusqu'à 64 clients et 32 parties simultanées avec une empreinte mémoire strictement déterministe, grâce à l'utilisation de pools d'objets pré-alloués.
— **Validation centralisée :** Le serveur agit comme le garant des règles. Il gère le cycle de vie des connexions via une machine à états à cinq phases, prévient les déconnexions silencieuses grâce à un *heartbeat*, et valide systématiquement la légalité des coups via le moteur pour empêcher toute triche.
— **Architecture réseau hybride :** Les communications exploitent deux protocoles complémentaires : l'UDP pour la découverte automatique des parties sur le réseau local, et le TCP pour assurer la transmission fiable des actions de jeu.

6.1.3 L'aboutissement visuel et l'expérience utilisateur

Enfin, l'interface graphique (GUI) vient valider le découpage modulaire du projet : elle s'est greffée au système sans nécessiter la moindre altération du code interne du moteur. La GUI couvre la totalité du cycle de jeu local avec une grande fluidité. Les joueurs bénéficient d'une navigation claire entre les pages, de fenêtres modales pour modifier les configurations (taille du plateau, modes IA, langue, raccourcis clavier) en pleine partie, et d'outils d'assistance visuelle comme l'indication des cases accessibles ou l'annulation de coups.

Le plateau, hautement interactif, supporte à la fois le clic et le glisser-déposer, s'adaptant ainsi aux préférences de l'utilisateur. L'ensemble est unifié par une charte graphique cohérente et intégralement paramétrable via une feuille de style CSS, prouvant que notre application est aussi aboutie sur le plan de l'utilisabilité que sur celui de l'ingénierie logicielle.

6.2 Limitations et améliorations possibles

6.2.1 Module IA : Les compromis de l'élagage et des heuristiques

Le développement de l'IA a mis en lumière la difficulté d'équilibrer la profondeur d'exploration et la pertinence de l'évaluation. Initialement, le Minimax classique sans élagage (NFP – *No Forward Pruning*) était limité à une profondeur de 2. Pour pallier cela, nous avons implémenté un *Forward Pruning* permettant d'atteindre la profondeur 3 (et théoriquement 5) dans un temps imparti.

Profondeur	**Sans *Forward Pruning* (ms)**	**Avec *Forward Pruning* (ms)**
1	1.19	1.25
2	612.94	121.59
3	10 630.93	255.94
4	-	2 950.79
5	-	7 559.63

TABLE 2 – Temps d'exécution de l'algorithme Minimax selon la profondeur et l'activation de l'élagage (*Forward Pruning*). CPU : Ryzen 7 Pro 7840U

Comme l'illustre le tableau 2, l'explosion combinatoire inhérente au Jeu des Amazones frappe l'algorithme brut de manière fulgurante. Sans élagage, le temps de calcul croît de manière strictement exponentielle, passant d'environ 1 ms à la profondeur 1 à plus de 10 secondes à la profondeur 3. Ce délai de 10,6 secondes représente un plafond de verre inacceptable pour garantir une expérience de jeu fluide et interactive, ce qui justifie notre limite initiale stricte à la profondeur 2 pour la version classique (*NFP*).

L'introduction du *Forward Pruning* bouleverse cette dynamique. Si l'on observe un infime surcoût de traitement à la profondeur 1 (1,25 ms contre 1,19 ms, lié au tri heuristique initial des nœuds), le bénéfice devient spectaculaire à mesure que l'arbre s'élargit. À la profondeur 3, le temps d'exécution est divisé par plus de 41 (passant de 10 630 ms à seulement 255 ms).

Cette réduction drastique de la complexité temporelle permet non seulement d'établir la profondeur 3 comme nouveau standard d'exploration instantanée, mais surtout de repousser la limite calculatoire de l'IA. L'algorithme parvient ainsi à atteindre la profondeur 5 en environ 7,5 secondes. Bien que coûteux, ce temps de réflexion reste largement inférieur à la limite théorique d'un tour de jeu (souvent fixée à 60 secondes en compétition), offrant ainsi au moteur un horizon tactique suffisant pour anticiper les pièges à moyen terme de l'adversaire.

6.2.2 Module IA : Les compromis de l'élagage et des heuristiques

Cependant, l'analyse de nos matrices d'affrontement (*Head-to-Head*) a révélé un comportement contre-intuitif sur des parties complètes : le Minimax profondeur 2 sans élagage (NFP) surpasse régulièrement les versions avec élagage à profondeur 3. Le tableau 3 illustre cette dynamique sur des séries de 50 duels directs.

Heuristique	Victoires NFP (Prof. 2)	Victoires FP (Prof. 3)	Taux de victoire NFP
QDKD	35	15	70 %
RTH	34	16	68 %
ME	27	23	54 %

TABLE 3 – Extrait des affrontements directs (50 parties par duel). L'exploration exhaustive à profondeur 2 l'emporte statistiquement sur l'exploration élaguée à profondeur 3.

Le problème : Le *pruning* a tendance à écarter prématurément des coups qui ne semblent pas rentables à court terme, mais qui s'avèrent être des coups de blocage critiques à long terme (effet d'horizon). Fait intéressant, lors de nos itérations, nous avons constaté qu'une de nos anciennes heuristiques d'évaluation rapide des coups (basée sur la distance au centre du plateau) compensait mieux certains angles morts du *pruning* que nos évaluations actuelles (évaluation du contrôle de jeu) en fonction de l'heuristique utilisée. Cela démontre l'extrême sensibilité de l'élagage vis-à-vis de la fonction d'évaluation choisie.

Pistes d'amélioration : Le *Forward Pruning* n'est pas pour autant une voie sans issue. Si ses performances globales sont mitigées en ouverture, nous avons observé qu'en milieu et fin de partie, la raréfaction des coups lui permet d'atteindre des profondeurs de 10, rendant l'IA redoutable (et gagnant avec QDKD contre MCTS). L'évolution logique serait une approche dynamique : utiliser un Minimax strict en début de partie pour ne rater aucun blocage, puis activer l'élagage profond lorsque le facteur de branchement diminue.

Enfin, l'intégration de tables de transposition (hachage Zobrist) pour éviter le recalcul des positions, la parallélisation du Minimax, ainsi que le renforcement de la capacité prédictive du MCTS constituent les prochaines étapes. Ce dernier point passerait par une complexification de l'architecture du réseau de neurones : l'ajout de **couches intermédiaires non linéaires** supplémentaires permettrait au modèle de mieux modéliser les abstractions tactiques du plateau en identifiant correctement des caractéristiques d'un plateau gagnant, le tout soutenu par un entraînement sur un volume de données plus massif (ici ; 100000 parties à MCTS 10ms vs MCTS 10ms).

6.2.3 Module réseau : Limites du mono-thread et persistance

Notre architecture événementielle autour de `poll()` garantit d'excellentes performances d'I/O, mais ce parti pris architectural induit certaines limites. Le modèle mono-threadé paralyserait l'intégralité du serveur s'il devait soudainement absorber des tâches bloquantes, comme le calcul centralisé d'une IA côté serveur. Sur le plan de la robustesse, la persistance du tableau des scores via un fichier texte simple reste vulnérable aux corruptions système, et le protocole ne gère pas nativement les reconnexions après une micro-coupure.

Pistes d'amélioration : Pour faire évoluer ce module vers un standard de production, la transition vers une base de données embarquée (`SQLite`) garantirait l'atomicité et la sécurité des sauvegardes. Pour pallier les limites du mono-thread, l'introduction d'un *pool* de *worker threads* permettrait de déléguer les calculs lourds sans ralentir la boucle réseau principale. L'expérience utilisateur serait grandement améliorée par un système de jetons de session (UUID) offrant une fenêtre de tolérance de reconnexion. Enfin, la sécurité des échanges devrait être assurée par une encapsulation TLS (via OpenSSL), et la bascule vers `epoll()` permettrait de dépasser la limite des 64 clients lors d'une montée en charge massive.

6.2.4 Module GUI : Intégration partielle et responsivité

Le développement de l'interface graphique a priorisé l'expérience de jeu local et la séparation avec le moteur. De ce fait, bien que les boutons d'hébergement réseau, de sauvegarde et de chargement soient présents visuellement, leurs gestionnaires de signaux n'ont pas encore été reliés aux couches inférieures. Le mode multijoueur reste donc actuellement exclusif à la CLI.

Sur le plan visuel, la taille des cellules du plateau a été statiquement fixée à 80×80 pixels. Si ce choix est parfait pour un format classique 10×10, il entraîne un débordement de la fenêtre sur les variantes de grandes dimensions.
Pistes d'amélioration : Le câblage des boutons réseau/sauvegarde vers la Factory IA et le noyau est trivial mais nécessite du temps de développement. L'amélioration visuelle principale consisterait à abandonner la taille fixe au profit d'un calcul dynamique des dimensions de la grille (*responsive design*), calculé proportionnellement à la géométrie de la fenêtre parente.

6.2.5 Moteur de jeu : Contraintes systèmes du mode Blitz

Le moteur remplit son cahier des charges de stabilité, mais une limite externe subsiste lors de l'exécution en terminal concernant le mode *Blitz*. La gestion du temps repose sur un calcul de différence (*delta*) entre deux actions. Or, la bibliothèque `readline` utilise des I/O bloquantes. Si un joueur dépasse son temps sans envoyer de frappe clavier, le processus reste en veille imposée par l'OS. Le moteur calcule bien le temps négatif en interne, mais l'événement de défaite ne se déclenche visuellement qu'au moment où l'utilisateur "réveille" le programme.

Pistes d'amélioration : Pour corriger ce comportement inhérent aux terminaux bloquants, la solution la plus robuste consisterait à intégrer des signaux POSIX (`SIGALRM`). Le système d'exploitation se chargerait d'envoyer un signal interrompant de force l'attente de `readline` dès que le chronomètre atteint zéro, permettant au moteur de déclarer la défaite en temps réel, de manière asynchrone et indépendamment des actions de l'utilisateur.

7 Transparence IA

Durant le projet, l'IA (Gemini 3.0 et Claude 4.5) a été utilisée à la manière d'un moteur de recherche, d'assistant de code, et également à but informatif. Des exemples d'utilisations sont :

- Résumé d'informations contenues dans des papiers de recherche ;
- Explication de concepts clés et vulgarisation scientifique (exemple : explication calculs Queen Distance & King Distance, ...) ;
- Vérification de logique directe d'architecture ;
- Exemples réduits de codes possibles, par rapport à d'autres scénario (exemple : pseudo-code MCTS pour comprendre le fonctionnement de son architecture) ;
- Aide à la mise en page du rapport (certaines commandes Latex) ;
- Réalisation de code "barebone" (code de base modifié à la suite).

Conclusion

La réalisation de ce projet en C nous a permis de mettre en pratique des concepts couvrant trois domaines complémentaires : la programmation système (gestion des sockets POSIX, multiplexage asynchrone, protocoles applicatifs), la programmation d'interfaces graphiques (GTK4, boucle d'événements GLib, glisser-déposer, CSS) et l'algorithmie appliquée à l'intelligence artificielle (Monte Carlo Tree Search, parallélisation multithread avec OpenMP, conception d'heuristiques). L'architecture en couches fortement découplées — moteur de jeu, réseau, CLI, GUI, IA — s'est révélée être un choix pertinent : le même moteur de jeu et la même API réseau ont été exploités par les deux interfaces ainsi que par le module IA sans modification. L'utilisation du patron Observateur pour les événements réseau a notamment permis de connecter la couche graphique au module réseau sans introduire de dépendance circulaire, tout comme l'étanchéité du moteur de jeu a permis à l'IA d'y brancher ses simulations de manière totalement isolée, ce qui constitue l'un des succès architecturaux majeurs de ce projet.

Références

[1] W. Zamkauskas. El juego de las amazonas. *El Acertijo*, 4, 1992.

[2] Timothy Furtak, Masashi Kiyomi, Takeaki Uno, and Michael Buro. Generalized amazons is pspace-complete. In *Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 132–137, 2005. URL : `https://www.ijcai.org/Proceedings/05/Papers/1300.pdf` [cited 2026-04-10].

[3] Richard J. Lorentz. Amazons discover monte-carlo. In *Computers and Games*, pages 13–24. Springer Berlin Heidelberg, 2008. `doi:10.1007/978-3-540-87608-3_2`.

[4] Tianhao Qian, Zhuoxuan Li, Jinde Cao, Xinli Shi, and Leszek Rutkowski. Resource-constrained amazons chess decision framework integrating large language models and graph attention, 2026. URL : `https://arxiv.org/abs/2603.10512` [cited 2026-04-10], `arXiv:2603.10512`.

[5] P. P. L. M. Hensgens. *A Knowledge-based Approach of the Game of Amazons*. Master's thesis, Universiteit Maastricht, Maastricht, Pays-Bas, September 2001. URL : `https://project.dke.maastrichtuniversity.nl/games/files/msc/Hensgens_thesis.pdf` [cited 2026-04-10].

[6] Louis Victor Allis. *Searching for Solutions in Games and Artificial Intelligence*. Phd thesis, University of Limburg, Maastricht, Pays-Bas, September 1994. URL : `http://fragrieu.free.fr/SearchingForSolutions.pdf` [cited 2026-04-10].

[7] John Tromp and Gunnar Farnebäck. Combinatorics of go. In H. Jaap van den Herik, Paolo Ciancarini, and H. H. L. M. (Jeroen) Donkers, editors, *Computers and Games*, pages 84–99, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg. `doi:10.1007/978-3-540-75538-8_8`.

[8] Jiaxing Song and Martin Müller. An enhanced solver for the game of amazons. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(1) :16–27, March 2015. `doi:10.1109/TCIAIG.2014.2309077`.

[9] Liang Qian. *Searching and Learning in the Game of Amazons*. Phd thesis, University of Alberta, Edmonton, AB, Canada, December 2015. URL : https://era.library.ualberta.ca/items/37279185-502a-4638-9646-6ec800a40090 [cited 2026-04-07].

[10] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *Machine Learning : ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, September 2006. doi:10.1007/11871842_29.

[11] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014. doi:10.3233/ICG-2014-37202.

[12] Chess Programming Wiki Contributors. Kogge-stone algorithm. Chess Programming Wiki, March 2025. URL : https://www.chessprogramming.org/Kogge-Stone_Algorithm [cited 2026-04-07].

[13] vishal_shevale. Alpha-beta pruning in adversarial search algorithms. GeeksforGeeks, January 2026. URL : https://www.geeksforgeeks.org/alpha-beta-pruning-in-adversarial-search-algorithms/ [cited 2026-04-07].

[14] Jens Lieberum. An evaluation function for the game of amazons. *Theoretical Computer Science*, 349(2) :230–244, December 2005. doi:10.1016/j.tcs.2005.09.048.

[15] kartik. Left-child right-sibling representation of tree. GeeksforGeeks, July 2025. URL : https://www.geeksforgeeks.org/left-child-right-sibling-representation-tree/ [cited 2026-04-07].

[16] Tristan Cazenave. Batch monte carlo tree search. In Cameron Browne and Akihiro Kishimoto, editors, *Computers and Games (CG 2022)*, pages 146–162. Springer, November 2022. doi:10.1007/978-3-031-34017-8_13.

[17] The GNOME Project. *GTK 4 Reference Manual*, 2026. URL : https://docs.gtk.org/gtk4/ [cited 2026-04-10].

Table des matières

1 Introduction **1**
- 1.1 Historique et héritage . . . 1
- 1.2 Règles . . . 1
- 1.3 Complexité . . . 3

2 Algorithmes et structures de données **3**
- 2.1 Moteur de jeu : algorithmes . . . 3
  - 2.1.1 Validation physique des déplacements (Algorithme de Raycasting) . . . 3
  - 2.1.2 Génération des coups possibles . . . 3
- 2.2 Le cœur du jeu et l'Intelligence Artificielle . . . 4
  - 2.2.1 Représentation de l'état et Complexité : les Bitboards . . . 4
  - 2.2.2 L'approche Minimax et ses heuristiques . . . 5
  - 2.2.3 Monte Carlo Tree Search (MCTS) et apprentissage automatique . . . 5
- 2.3 Réseau : structures de données et protocole . . . 6
  - 2.3.1 Le problème du *Framing* TCP . . . 6
  - 2.3.2 Protocole applicatif et sérialisation . . . 7
- 2.4 Moteur de jeu : structures de données . . . 8
  - 2.4.1 Modélisation matricielle de l'état du jeu . . . 8
  - 2.4.2 Architecture Bitboard . . . 8
- 2.5 Interface graphique : structures de données . . . 8
  - 2.5.1 Le contexte applicatif `GuiContext` . . . 8
  - 2.5.2 L'automate des phases de jeu . . . 8
  - 2.5.3 Les raccourcis clavier `KeyBindings` . . . 8

3 Architecture du projet **9**
- 3.1 Découpage en modules . . . 9
- 3.2 Résumé de l'architecture . . . 9
- 3.3 Architecture détaillée du module IA . . . 10
  - 3.3.1 Isolation totale et interface unifiée . . . 10
  - 3.3.2 Le patron Factory : découplage par chaîne de caractères . . . 11
  - 3.3.3 Injection de configuration par dictionnaire dynamique . . . 11
  - 3.3.4 Récursion du patron : Evaluators et Selectors . . . 11
  - 3.3.5 Bilan : une architecture en poupées russes . . . 12
  - 3.3.6 Modélisation matricielle et source de vérité . . . 12
  - 3.3.7 Indépendance du code et encapsulation de la complexité . . . 12
  - 3.3.8 Programmation défensive et sécurité . . . 13
  - 3.3.9 Architecture hybride C/Python pour l'Apprentissage Automatique . . . 13
- 3.4 Architecture du module réseau . . . 14
  - 3.4.1 La façade `NetworkManager` . . . 14
  - 3.4.2 Le multiplexage asynchrone avec `poll()` . . . 14
  - 3.4.3 Le patron Observateur pour le découplage . . . 15
  - 3.4.4 Routage des paquets et arbitrage serveur . . . 15
  - 3.4.5 Structures de contexte et gestion des connexions . . . 16
  - 3.4.6 Sérialisation et framing TCP . . . 16
- 3.5 Le module d'interface graphique (GUI) . . . 17
  - 3.5.1 Séparation déclarative et encapsulation des ressources . . . 17
  - 3.5.2 L'orchestrateur asynchrone et la Main Loop . . . 17
  - 3.5.3 Automate fini interactif et approche transactionnelle . . . 17
  - 3.5.4 Navigation lâche et stylisation dynamique . . . 17
- 3.6 Architecture du module CLI . . . 18
  - 3.6.1 Le contexte CLI (`CliContext`) et le routage centralisé . . . 18
  - 3.6.2 Boucle événementielle hybride et multiplexage . . . 18
  - 3.6.3 Sérialisation et robustesse du parsing . . . 18
  - 3.6.4 Validation mathématique et sécurité des entrées . . . 18

4 Explication approfondie **19**
4.1 Fonctionnalité réseau : Système d'invitations et machine à états (F40) . . . 19
4.1.1 Prérequis et problème . . . 19
4.1.2 Solution implémentée : machine à états finis . . . 19
4.1.3 Garanties de cohérence . . . 19
4.2 Fonctionnalité 2 : Approfondissement itératif avec contrainte de temps . . . 20
4.2.1 Prérequis et problème . . . 20
4.2.2 Solution implémentée : Iterative Deepening . . . 20
4.2.3 Intégration avec Forward Pruning (F33) . . . 20

5 Stratégie de test et assurance qualité **21**
5.1 Framework de test : GoogleTest . . . 21
5.2 Infrastructure de test : fixtures et mocks . . . 21
5.3 Couverture de code avec `gcovr` . . . 21
5.4 Analyse statique avec `clang-tidy` . . . 22
5.5 Conformité de style avec `clang-format` . . . 22
5.6 Tentative de fuzzing . . . 22

6 Revue critique du projet **22**
6.1 Ce qui a été accompli : une architecture synergique et robuste . . . 22
6.1.1 Un socle logique performant au service de l'IA . . . 22
6.1.2 Résilience, sécurité et intégrité multijoueur . . . 22
6.1.3 L'aboutissement visuel et l'expérience utilisateur . . . 23
6.2 Limitations et améliorations possibles . . . 23
6.2.1 Module IA : Les compromis de l'élagage et des heuristiques . . . 23
6.2.2 Module IA : Les compromis de l'élagage et des heuristiques . . . 24
6.2.3 Module réseau : Limites du mono-thread et persistance . . . 24
6.2.4 Module GUI : Intégration partielle et responsivité . . . 24
6.2.5 Moteur de jeu : Contraintes systèmes du mode Blitz . . . 25

7 Transparence IA **25**

A Liste des besoins généraux **29**

B Liste des besoins gestion du jeu **30**

C Liste des besoins CLI **30**

D Liste des besoins format de sauvegarde **31**

E Liste des besoins structures de données **31**

F Liste des besoins GUI **32**

G Liste des besoins intelligence artificielle **33**

H Liste des besoins réseau **34**

I Protocole réseau : types de paquets **37**

A Liste des besoins généraux

Exigence CDC	Compl.	Implémentation
F1–F3. Lancement et configuration du programme		
F1. Lancement via exécutable `amazons [OPTIONS]`	**60%**	Exécutable `amazons` compilé via CMake ; options `-h`, `-V` implémentées dans `main.c`. -v définit bien la verbosité, mais n'est pas utilisé. -d est raccourci en -v ; et n'a pas son propre cas mis en place.
F2. Fichier de configuration `.amazonsrc` au format INI	**80%**	`config.c` lit/écrit le fichier INI ; sections `[defaults]` ; options CLI supplantent la config. Cependant, la lecture du fichier ne se fait pas réellement comme un fichier ini.
F3. Internationalisation (i18n) anglais/français	**75%**	`gettext` avec fichier `amazons.pot` ; détection via `LANG/LC_ALL` mais que coté CLI. ; fallback anglais.

TABLE 4 – F1–F3 : Lancement et configuration — **71%**

Exigence CDC	Compl.	Implémentation
F4–F6. Options de lancement		
F4. Interface CLI par défaut, chargement fichier en argument	**70%**	`cli_controller.c` lance le shell interactif ; cependant, si le fichier est simplement donné en argument, la partie n'est pas chargée.
F5. Mode blitz avec `-b/-blitz` et `-t TIME/-time TIME`	**90%**	Variables `white_time_left`, `black_time_left` ; `update_blitz_timer()` appelé à chaque coup ; défaut 30 min. Ne met pas de message d'avertissement si `-t` sans `-b`
F6. Mode contest avec `-c/-contest`	**20%**	Option prise en compte en config, mais non implémenté.

TABLE 5 – F4–F6 : Options de lancement — **60%**

Exigence CDC	Compl.	Implémentation
F7–F9. Modes de jeu		
F7. Interface graphique avec `-g/-gui`	**90%**	`gui_application.c` avec GTK4 ; mêmes fonctionnalités que CLI via fenêtre graphique. Cependant : s'il n'y a pas d'écran disponible, le programme plante au lieu de fallback en CLI // d'être plus expressif sur l'erreur.
F8. Mode IA avec `-a [COLOR]/-ai [COLOR]`	**75%**	Les options sont nommées –ai-white et –ai-black, afin de pouvoir spécifier des paramètres différents pour chacune des IAs.
F9. Taille plateau avec `-n SIZE/-size SIZE` (4–11)	**100%**	`board_size` configurable ; validation `MIN_BOARD_SIZE` à `MAX_BOARD_SIZE` ; défaut 10.

TABLE 6 – F7–F9 : Modes de jeu — **88%**

B Liste des besoins gestion du jeu

Exigence CDC	Compl.	Implémentation
F10–F14. Gestion des règles et de la partie		
F10. Gestion des règles : validation coups, détection fin de partie	**100%**	`is_move_legal()` valide via bitboards ; `is_game_over()` détecte blocage ; `get_winner()` détermine vainqueur.
F11. Gestion des erreurs utilisateur avec messages explicites	**100%**	Coups illégaux rejetés par le moteur ; contrôleur affiche message d'erreur localisé.
F12. Undo/redo avec historique des coups	**100%**	Piles LIFO `undo_stack`/`redo_stack` ; `undo_move()` et `redo_move()` avec inversion atomique ; annulation jusqu'au joueur humain précédent.
F13. Conseils de coup via commande `hint`	**100%**	`sync_from_logic()` expose l'état au module IA ; meilleur coup affiché en notation algébrique.
F14. Gestion du blitz : affichage temps, pause, arrêt partie	**50%**	Affichage temps restant et arrêt automatique implémentés ; **mise en pause non implémentée**, le temps n'est pas décompté pendant le tour du joueur mais après.

TABLE 7 – F10–F14 : Gestion des règles — **90%**

C Liste des besoins CLI

Exigence CDC	Compl.	Implémentation
F15–F17. Shell interactif		
F15. Shell avec prompt, commandes `new`, `help`, `quit`, `load`, `save`, `pause`, `hint`, `undo`, `redo`, `show`, `set`	**70%**	`cli_commands.c` implémente toutes les commandes ; dispatch via table de fonctions . Cependant, certains soucis/oublis : hint est utilisable au menu principal, set debug=true ne marche pas (F1 - debug n'est pas initialisé séparément), et load_game charge bien la partie, mais ne la lance pas (ne quitte pas le prompt.) .
F16. Proposer sauvegarde avant `quit` si non sauvegardé	**100%**	Dialogue `Save the game before quitting? [y/N]` ; redemande en cas d'erreur d'enregistrement. Ne marche qu'en local (non en réseau).
F17. Édition ligne de commande (insertion, suppression, curseur)	**100%**	GNU Readline intégré via `input.c` ; navigation curseur avec flèches.

TABLE 8 – F15–F17 : Shell interactif — **90%**

Exigence CDC	Compl.	Implémentation
F18–F20. Historique et notation		
F18. Historique commandes avec flèches et `<Ctrl>+<R>`	**100%**	GNU Readline gère l'historique ; navigation haut/-bas ; recherche incrémentale.
F19. Complétion commandes avec `<Tab>`	**100%**	`rl_completion_entry_function` configuré ; double-tab affiche alternatives.
F20. Notation algébrique `e2-e4/e6` (colonnes a–k, lignes 1–11)	**100%**	Parsing dans `cli_commands.c` ; validation coordonnées ; message si coup illégal.

TABLE 9 – F18–F20 : Historique et notation — **100%**

D Liste des besoins format de sauvegarde

Exigence CDC	Compl.	Implémentation
F21–F25. Format de sauvegarde		
F21. Sauvegarde/restauration au format ASCII avec gestion erreurs	**95%**	`save_game()` et `load_game` fonctionnent et gèrent les cas d'erreurs.
F22. Commentaires ligne (`#`) et bloc (`{}`)	**100%**	`clean_line()` filtre commentaires ; `in_block_comment` gère les blocs multi-lignes.
F23. Section `[settings]` avec paramètres clé=valeur	**80%**	`write_settings()` et `parse_settings_line()` ; debug, erreur temps restant du mode blitz non pris en compte, ai-mode, etc.
F24. Section `[game]` avec plateau ASCII (Q/q/X/_) et joueur courant (W/B)	**95%**	`write_game_state()` et `parse_board_line()` ; format conforme à la spec.
F25. Section `[history]` avec coups en notation algébrique	**70%**	`write_history()` et `parse_history_line()` ; format `e2-e4/e6`. ne vérifie pas que les coups de l'historique ont bien été fait

TABLE 10 – F21–F25 : Format de sauvegarde — **63%**

E Liste des besoins structures de données

Exigence CDC	Compl.	Implémentation
F26–F27. Module bitboard		
F26. Module bitboard indépendant pour représenter l'état du plateau	**90%**	Structure `Bitboard` 128 bits (`low/high`) ; `BitboardState` avec reines et flèches ; module isolé dans `bitboard.c/bitboard.h`.Ne prend pas les cas nuls en compte par soucis de performance
F27. Algorithmes bitboard : validation coups, calcul coups possibles, application coup, scores, fin de partie	**100%**	Kogge-Stone pour trajectoires ; `expand_queen_slide()/expand_king_step()` ; masques pré-calculés `MASK_NOT_A/MASK_NOT_H` ; `bitboard_state_is_game_over()`.

TABLE 11 – F26–F27 : Module bitboard — **95%**

F Liste des besoins GUI

Exigence CDC	Compl.	Implémentation
F28–F30. Interface graphique		
F28. Menus File (New, Load, Save, Config, Info, Quit) et Game (Undo, Redo, Pause, Hint)	**90%**	Menu principal avec 5 boutons ; popups Config et Info fonctionnels. Pause non implémentée, Placeholder pour la version du logiciel dans la page et dans le popup Info.
F29. Raccourcis clavier configurables (10 actions)	**90%**	`GtkShortcutController` avec actions `Ctrl+N/L/S/,/I/Q/U/R/P/H` ; personnalisation via boutons toggle ; sauvegarde dans config. Raccourci pour pause fonctionnel, mais ne fait rien
F30. Glisser-déposer pour jouer à la souris	**100%**	`GtkDragSource` + `GtkDropTarget` sur chaque cellule ; `onDrag()`/`onDrop()` gèrent le déplacement ; tir flèche par clic.

TABLE 12 – F28–F30 : Interface graphique — **97%**

Fonctionnalité GUI	Compl.	Implémentation
Fonctionnalités GUI complémentaires		
Plateau interactif dans `GtkGrid`	**100%**	`setupBoard()` alloue $n \times n$ cellules ; `drawBoard()` met à jour le rendu.
Mise en évidence des cases accessibles	**100%**	Calcul via `bitboard_get_reachable()` ; classe CSS `highlight-valid`.
Annulation déplacement par clic droit	**100%**	`GtkGestureClick` secondaire ; `onRightClick()` restaure l'état.
Intégration IA asynchrone	**100%**	Tour IA planifié sans bloquer GTK ; `guiHandlePlayerMove()` enchaîne.
Undo/redo double coup (joueur + IA)	**100%**	`undoButtonClick()` appelle `undo_move()` deux fois ; uniquement en mode IA.
Charte graphique violet/or	**100%**	`style.css` chargé via `GtkCssProvider`.
Compatibilité GTK4 $\geq$ 4.10	**100%**	Blocs `#if GTK_CHECK_VERSION` pour `GtkAlertDialog`/`GtkDropDown`.
Libération ressources à la fermeture	**100%**	`freeAppContext()` libère plateau et appelle `game_cleanup()`.

TABLE 13 – Fonctionnalités GUI complémentaires — **100%**

G Liste des besoins intelligence artificielle

Exigence CDC	Compl.	Implémentation
F31–F33. Algorithme Minimax		
F31. Temps de réflexion borné avec `-ai-time TIME`; retour meilleur coup trouvé (défaut 5s)	**100%**	`time_limit` vérifié via `get_time_ms()` dans `alpha_beta_conditions()` et `run_root_search()`; `DEFAULT_TIME_LIMIT = 5000`.
F32. Minimax avec $\alpha\beta$-pruning via `-ai-mode minimax`	**90%**	Coupure `beta <= alpha` dans `alpha_beta_forward()`; `update_best_move()` met à jour bornes.Il peut y avoir certain cas avec null non traités.
F33. Au moins 3 fonctions d'évaluation via `-ai-minimax-scoring`	**100%**	`eval_me()` (mobilité), `eval_rth()` (territoire relatif BFS), `eval_qdkd()` (distance reine/roi combinée); sélection via `get_evaluator()`.

TABLE 14 – F31–F33 : Algorithme Minimax — **96%**

Exigence CDC	Compl.	Implémentation
F34–F35. Minimax avancé		
F34. Approfondissement itératif via `-ai-mode iterative`	**100%**	Boucle `for (d = 1; d <= max_depth; d++)` avec conservation meilleur coup; tri via `sort_moves()` pour améliorer pruning.
F35. Profondeur configurable via `-ai-minimax-depth DEPTH`	**80%**	Paramètre `max_depth` initialisé via `ConfigMap`; `DEFAULT_MAX_DEPTH = 3`; profondeur auto si non spécifiée fonctionnel en iterative deepening, mais non fonctionnel hors iterative deepening; le programme s'attend à atteindre sa profondeur spécifiée et renverra donc le premier coup trouvé à la place.

TABLE 15 – F34–F35 : Minimax avancé — **90%**

Exigence CDC	Compl.	Implémentation
F36–F37. MCTS et Machine Learning		
F36. MCTS avec UCT via `-ai-mode mcts`	**90%**	`select_node()` avec interface `ISelector`; `expand()`, `rollout()`, `backpropagate()`; UCT avec $C = \sqrt{2}$; support multi-threading OpenMP mis en place mais non testé rigoureusement.
F37. Fonction de sélection ML via `-ai-mcts-selection ML`	**60%**	Architecture `ISelector` extensible; intégration TensorFlow C API prévue; modèle entraîné; fonctionnalité expérimentale (assez peu testée).

TABLE 16 – F36–F37 : MCTS et Machine Learning — **75%**

Fonctionnalité bonus	Compl.	Implémentation
Améliorations IA au-delà du cahier des charges		
Forward pruning pour réduction espace de recherche	**100%**	`get_beam_width()` calcule largeur faisceau selon profondeur ; `get_top_moves()` filtre coups prometteurs.
Parallélisation MCTS multi-threads	**100%**	Arbres indépendants par thread via `#pragma omp parallel` ; fusion résultats via `merge_and_get_best()`.
Rollout heuristique avec évaluation anticipée	**100%**	`rollout()` coupe à profondeur 5 ; `get_mcts_win_probability()` estime probabilité victoire.
Factory pattern pour instanciation IA	**100%**	`ai_factory.c` crée Minimax ou MCTS selon config ; extensible pour nouveaux algorithmes.

TABLE 17 – Fonctionnalités bonus IA

H Liste des besoins réseau

Exigence CDC (F38)	Compl.	Implémentation
F38. Découverte de serveurs et connexion client-serveur basique		
`server start [PORT]` : Démarrage serveur TCP sur port configurable (12345 par défaut), diffusion UDP de présence	**85%**	`NetworkManager_ServerStart()` ; option `-s [PORT]` ; broadcast UDP toutes les 10 s sur port 12346. On peut lancer sur n'importe quel port de plus de 1024 (erreur pas de maximum.
`server list` : Liste des serveurs via UDP broadcast, expiration après 30 s sans signal	**100%**	`ClientDiscoveryContext` écoute le port UDP ; `NetworkManager_GetServerList()` filtre les entrées expirées après 30 s.
`join [IP[:PORT]]` : Connexion au serveur avec gestion d'erreur explicite	**100%**	`NetworkManager_ClientJoin()` + `NetworkManager_SendJoin()` ; messages d'erreur si connexion échoue.
`server stop` : Arrêt du serveur avec notification des clients connectés	**100%**	`NetworkManager_ShutdownServer()` envoie `S2C_ERROR` à tous les clients avant nettoyage.
`quit` : Déconnexion propre du client, retour en mode local	**100%**	`execute_quit()` appelle `NetworkManager_Cleanup()` ; retour au menu principal.
`ping` : Test de connexion avec affichage du temps de réponse	**100%**	Horodatage en ms dans le payload de `C2S_PING` ; RTT calculé à réception de `S2C_PONG`.
Timeout de déconnexion (1 minute) pour gérer les déconnexions inopinées	**100%**	Boucle de vérification dans `handle_server_tick()` ; kick après 60 s d'inactivité. Heartbeat silencieux toutes les 30 s pour les clients légitimes inactifs.
Messages ASCII terminés par `'\n'`	**100%**	Format `"TYPE|PAYLOAD\n"` ; `SerializePacket()` / `DeserializePacket()`.

TABLE 18 – F38 : Découverte et connexion basique — **98%**

Exigence CDC (F39)	Compl.	Implémentation
F39. Serveur multi-clients et gestion de parties		
Gestion multi-clients simultanés (thread par client ou asynchrone)	**100%**	Modèle événementiel mono-threadé avec `poll()`; jusqu'à 64 clients simultanés sans threads.
Protection des données partagées (mutex/verrous)	**100%**	Architecture mono-threadée élimine les accès concurrents; aucun mutex nécessaire par conception.
`server status` : Port d'écoute, nombre de clients, parties en cours	**75%**	`NetworkManager_GetServerStatus()` fonctionne en mode serveur local; non exposé via paquet réseau pour un client distant.
`players` : Liste des joueurs (id, nom, statut)	**100%**	`NetworkManager_GetServerPlayerList()`; paquets `C2S_LIST` / `S2C_LIST`.
`scoreboard` : Tableau des scores persistant (victoires, défaites, parties)	**95%**	Fichier `scoreboard.txt` géré par `network_stats.c`; paquets `C2S_SCOREBOARD` / `S2C_SCOREBOARD`. Le scoreboard ne s'affiche pas bien sur le serveur
Routage des coups vers la bonne partie	**100%**	`process_move_packet()` identifie l'adversaire via `game_id` partagé et route le coup.
Validation des coups côté serveur (anti-triche)	**100%**	`is_move_legal()` appelé sur copie locale du plateau avant tout relais; coup illégal rejeté avec `S2C_ERROR`.
Option `-s [PORT]` / `-server [PORT]`	**100%**	Traitée dans `main`; lance directement en mode serveur.
Option `-d` / `-daemon` : Mode headless	**100%**	Appel à `daemon(0,0)` sous POSIX; serveur sans interface.

TABLE 19 – F39 : Serveur multi-clients — **96%**

Exigence CDC (F40)	Compl.	Implémentation
F40. Système d'invitations et gestion avancée des joueurs		
`players [PLAYER_ID]` : Informations détaillées d'un joueur spécifique	**100%**	Branche `target_id != -1` dans `NetworkManager_GetServerPlayerList()`.
Statuts de joueur : `idle`, `away`, `waitgame`, `ingame`	**80%**	Énumération `ClientStatus` à 5 états ; transitions protégées par garde-fous.
`new PLAYER_ID` : Envoi d'invitation avec notification	**100%**	`process_invite_packet()` vérifie disponibilité ; `S2C_INVITE` envoyé à la cible.
`accept` : Acceptation avec démarrage immédiat	**100%**	`process_accept_packet()` crée le jeu et envoie `S2C_GAME_START` aux deux joueurs.
`decline` : Refus avec notification de l'inviteur	**100%**	`process_decline_packet()` libère les deux joueurs ; `S2C_DECLINE` envoyé.
`cancel` : Annulation d'invitation en attente	**100%**	Paquet `C2S_CANCEL` ; le serveur notifie la cible via `S2C_CANCEL` et libère l'inviteur.
`away` / `back` : Changement de disponibilité	**70%**	Paquets `C2S_AWAY` / `C2S_BACK` ; transitions `IDLE` $\leftrightarrow$ `AWAY`. Cas non pris en compte quand on cancel une invitation en étant away (on n'est pas prévenu du changement d'état)
Timeout d'invitation (5 minutes) avec expiration automatique	**100%**	Boucle de vérification dans `handle_server_tick()` ; expiration après 300 s avec notification des deux joueurs.
Empêcher actions invalides (inviter joueur en partie, accepter sans invitation, etc.)	**100%**	Vérifications systématiques : `pending_invite_from == -1`, statut cible $\neq$ `INGAME/AWAY`, etc. ; `S2C_ERROR` explicite.
Validation unicité du pseudo à la connexion	**100%**	`process_join_packet()` vérifie l'unicité avant d'accepter ; `S2C_REJECT` si doublon.

TABLE 20 – F40 : Système d'invitations — **95%**

Fonctionnalité bonus	Compl.	Implémentation
Améliorations réseau au-delà du cahier des charges		
Heartbeat silencieux pour maintenir les connexions inactives légitimes	**100%**	Paquet `C2S_HEARTBEAT` émis toutes les 30 s ; permet aux joueurs `AWAY` de rester connectés.
Gestion du timeout de tour (60 s) avec notification réseau	**100%**	Timer dans la boucle CLI ; paquet `PKT_TIMEOUT` routé par le serveur à l'adversaire.
Victoire par forfait lors de déconnexion adverse	**100%**	`disconnect_server_client()` met à jour les statistiques et envoie `S2C_OPPONENT_LEFT`.
Empreinte mémoire déterministe et bornée	**100%**	Slots de taille fixe ; aucune allocation dynamique en cours de fonctionnement.

TABLE 21 – Fonctionnalités bonus réseau

Exigence du cahier des charges	Complétion	Commentaire
F38 : Découverte et connexion basique	**100%**	Toutes les commandes implémentées
F39 : Serveur multi-clients et gestion de parties	**97%**	`server status` limité au mode local
F40 : Système d'invitations et gestion avancée	**100%**	Machine à états complète
Total module réseau	**99%**	—

TABLE 22 – Synthèse de la couverture des exigences réseau

I Protocole réseau : types de paquets

Type	Direction	Description
Authentification et session		
`C2S_JOIN`	C → S	Demande de connexion avec pseudo
`S2C_WELCOME`	S → C	Confirmation d'acceptation du pseudo
`S2C_REJECT`	S → C	Rejet du pseudo (déjà utilisé)
`C2S_HEARTBEAT`	C → S	Maintien de connexion silencieux
`C2S_PING`	C → S	Test de latence (horodatage inclus)
`S2C_PONG`	S → C	Réponse au ping
Gestion du lobby		
`C2S_LIST`	C → S	Demande de liste des joueurs
`S2C_LIST`	S → C	Liste des joueurs connectés
`C2S_AWAY`	C → S	Passage en statut indisponible
`C2S_BACK`	C → S	Retour en statut disponible
`C2S_SCOREBOARD`	C → S	Demande du tableau des scores
`S2C_SCOREBOARD`	S → C	Ligne du tableau des scores
Système d'invitations		
`C2S_INVITE`	C → S	Invitation avec taille de plateau
`S2C_INVITE`	S → C	Notification d'invitation reçue
`C2S_ACCEPT`	C → S	Acceptation d'invitation
`C2S_DECLINE`	C → S	Refus d'invitation
`S2C_DECLINE`	S → C	Notification de refus
`C2S_CANCEL`	C → S	Annulation d'invitation émise
`S2C_CANCEL`	S → C	Notification d'annulation
Déroulement de la partie		
`S2C_GAME_START`	S → C	Démarrage avec rôle assigné
`C2S_MOVE`	C → S	Envoi d'un coup joué
`S2C_MOVE`	S → C	Relais du coup adverse
`PKT_TIMEOUT`	Bidir.	Signalement d'expiration de tour
`C2S_GAME_OVER`	C → S	Déclaration de fin de partie
`S2C_OPPONENT_LEFT`	S → C	Déconnexion de l'adversaire
`S2C_ERROR`	S → C	Message d'erreur générique

TABLE 23 – Liste complète des 27 types de paquets du protocole. C = Client, S = Serveur, Bidir. = Bidirectionnel.